

Exploration des concepts et des techniques
d'apprentissage par renforcement : De la théorie à la
pratique

TRAVAIL DE MASTER RÉALISÉ EN VUE DE
L'OBTENTION DU MASTER OF SCIENCE HES-SO EN
SCIENCES DE L'INFORMATION PAR :

DAVID PAULINO

CONSEILLERS AU TRAVAIL DE MASTER :

PR ALEXANDROS KALOUSIS

GENÈVE, LE 6 JUILLET 2026

Haute école de Gestion de Genève (HEG-GE)

Filière Information Science

1 Déclaration

Ce travail de Master est réalisé dans le cadre du Master en Sciences de l'information de la Haute école de gestion de Genève.

L'étudiant-e atteste que le travail rendu est le fruit de sa réflexion personnelle, a été rédigé de manière autonome sans avoir utilisé des sources autres que celles citées dans la bibliographie et a été vérifié par un logiciel de détection de plagiat.

L'étudiant-e accepte, le cas échéant, la clause de confidentialité.

L'utilisation des conclusions et recommandations formulées dans ce travail, sans préjuger de leur valeur, n'engage ni la responsabilité de l'étudiant-e, ni celle du/de la directeur-trice.

Fait à Genève, le 6 juillet 2026

David Paulino

2 Remerciements

La partie où on remercie les gens qui nous ont aidé à réaliser ce travail de Master.

3 Résumé

Voilà le résumé de ce travail de Master. Il est important de bien résumer le travail pour que le lecteur puisse comprendre le contenu de ce travail sans avoir à le lire en entier.

Liste des tableaux

1	Comparaison des algorithmes DQN, PPO et SAC selon leurs caractéristiques fondamentales.	31
2	Hyperparamètres testés pour DQN	40
3	Hyperparamètres testés pour PPO	41
4	Hyperparamètres testés pour SAC	42
5	Score Optuna des meilleures configurations (récompense moyenne sur 20 épisodes déterministes, meilleur essai parmi 150)	44
6	Hyperparamètres optimaux — DQN / discret (score Optuna : 275.14)	44
7	Hyperparamètres optimaux — PPO / discret (score Optuna : 278.76)	45
8	Hyperparamètres optimaux — PPO / continu (score Optuna : 280.42)	46
9	Hyperparamètres optimaux — SAC / continu (score Optuna : 284.05)	47
10	Résultats de l'évaluation multi-seeds (500 000 steps, récompense moyenne sur 20 épisodes déterministes, écart-type échantillonnal $n - 1$)	50
11	Espace d'actions dans RLGym (RLGym, 2020b)	54
12	Exemples de <i>reward hacking</i> observés et corrections conceptuelles apportées	62
13	Curriculum de récompenses utilisé pour la comparaison PPO/SAC	63

Table des figures

1	Boucle d'interaction agent-environnement en apprentissage par renforcement	12
2	Environnement Cliff Walking.	12
3	Itérations de l'évaluation de politique dans un monde de grille 4×4 . Les états terminaux sont grisés. Inspiré de Silver (2015).	16
4	Aperçu de l'environnement LunarLander	37
5	Distribution du temps de calcul (en minutes) pour chaque combinaison algorithme/environnement, sur les 5 seeds indépendantes.	51
6	Consommation du boost lorsque la vitesse supersonique est atteinte	70
7	Collecte du boost sur le terrain	70
8	Comparaison du temps passé au sol et dans les airs par l'agent PPO au cours d'un match contre un joueur humain de niveau Champion	71
9	Heatmap de positionnement comparée entre l'adversaire humain (à gauche) et l'agent PPO (à droite) au cours d'un match contre un joueur humain de niveau Champion	72
10	Utilisation du dérapage par l'agent PPO et par un joueur humain de niveau Champion au cours d'un même match	73

Table des matières

1	Déclaration	1
2	Remerciements	2
3	Résumé	3
	Liste des tableaux	4
	Table des figures	5
4	Introduction	10
5	État de l’art	11
5.1	Le paradigme Agent-Environnement	11
5.1.1	Exemple : Cliff Walking	12
5.2	Spécificités et Concepts Clés	13
5.3	Processus Décisionnel de Markov (MDP)	14
5.4	Model-Based	15
5.4.1	Policy Evaluation et Policy Iteration	15
5.4.2	Value Iteration	17
5.5	Model-Free	17
5.5.1	Monte-Carlo	18
5.5.2	Temporal-Difference (TD)	19
5.6	Apprentissage en environnement inconnu	22

5.6.1	On-Policy vs Off-Policy	22
5.6.2	Le dilemme Exploration-Exploitation	22
5.6.3	SARSA	23
5.6.4	Q-Learning	24
5.7	Du RL Tabulaire au Deep RL	24
5.7.1	Le défi de la stabilité	24
5.7.2	Deep Q-Network	25
5.8	Méthodes Actor-Critic	26
5.8.1	Architecture Actor-Critic	26
5.8.2	Proximal Policy Optimization (PPO)	27
5.8.3	Soft Actor-Critic (SAC)	29
5.9	Synthèse et positionnement des algorithmes étudiés	31
6	Problématique	32
7	Outils utilisés	33
7.1	Gymnasium	33
7.2	Stable Baselines3	33
7.3	Optuna	34
7.4	Weights & Biases	34
7.5	RLGym	35
7.5.1	RocketSim	35
7.5.2	RocketSimVis	35

7.5.3	RLBot	35
7.5.4	RLGym-PPO	36
7.5.5	RLGym-SAC	36
8	Phase I - Validation méthodologique	36
8.1	Environnement et algorithmes sélectionnés	37
8.2	Optimisation des hyperparamètres	38
8.2.1	Cadre général	38
8.2.2	Élagage des essais non concluants	39
8.2.3	Hyperparamètres testés	39
8.3	Résultats de l'optimisation	43
8.3.1	DQN — LunarLander discret	44
8.3.2	PPO — LunarLander discret	45
8.3.3	PPO — LunarLanderContinuous	46
8.3.4	SAC — LunarLanderContinuous	47
8.4	Évaluation finale multi-seeds	48
8.4.1	Protocole d'évaluation finale	48
8.4.2	Résultats de l'évaluation multi-seeds	49
9	Phase II - Apprentissage sur Rocket League	52
9.1	Environnement de jeu	52
9.1.1	Espace d'actions	53
9.1.2	Espace d'observation	56

9.2	Objectif du jeu et problème du reward sparse	57
9.3	Reward shaping	59
9.4	Curriculum d'apprentissage	60
9.5	Reward hacking et limites du reward shaping	61
9.6	Méthodologie de comparaison entre PPO et SAC	63
9.6.1	Phase I – Contrôle de base et contact avec la balle . . .	63
9.6.2	Phase II – Apprentissage du score	64
9.6.3	Phase III – Perfectionnement mécanique	64
9.6.4	Limites d'implémentation rencontrées avec SAC	65
9.6.5	Résultats et divergence de convergence entre PPO et SAC	66
9.6.6	Discussion : effet de l'espace d'actions continu vs discret	67
9.7	Résultats avec PPO	68
10	Discussion transversale	73
11	Conclusion	73

4 Introduction

Parmi les différents paradigmes de l'intelligence artificielle, l'apprentissage par renforcement (*Reinforcement Learning* ou RL) se distingue par sa capacité à former des agents décisionnels autonomes par le biais d'une interaction continue avec leur environnement (Sutton et al., 2018). Contrairement à l'apprentissage supervisé qui s'appuie sur des ensembles de données étiquetés, le RL repose fondamentalement sur un processus d'essais et d'erreurs. L'agent explore son environnement et affine sa *politique* d'action en se basant exclusivement sur des signaux de récompense, avec pour objectif ultime de maximiser ses retours cumulés sur le long terme.

Cette approche inspirée de l'apprentissage animal a démontré des résultats spectaculaires dans des domaines aussi variés que la maîtrise de jeux stratégiques complexes (Silver et al., 2017; OpenAI et al., 2019), le contrôle en robotique (Tang et al., 2024) ou encore la personnalisation des systèmes de recommandation (Iftikhar et al., 2024). Toutefois, son déploiement pose d'importants défis pratiques, notamment en ce qui concerne l'efficacité d'échantillonnage (*sample efficiency*), la formulation mathématique pertinente de la fonction de récompense, et la gestion du délicat compromis entre l'exploration de nouvelles actions et l'exploitation des connaissances déjà acquises.

5 État de l'art

Cette section introduit les fondations théoriques de l'apprentissage par renforcement, en s'appuyant principalement sur l'ouvrage de référence de Sutton et al. (2018) et les cours de Silver (2015).

5.1 Le paradigme Agent-Environnement

Le socle du RL est modélisé par une boucle d'interaction perpétuelle entre deux entités distinctes (Sutton et al., 2018) (voir Figure 1) :

- **L'agent** : le système apprenant et décisionnel.
- **L'environnement** : le système externe avec lequel l'agent interagit et qui réagit à ses actions.

À chaque pas de temps discret t , ce cycle se déroule comme suit :

1. L'agent perçoit une **observation** O_t de l'environnement et reçoit un signal scalaire de **récompense** R_t .
2. Sur la base de ces informations, l'agent sélectionne et exécute une **action** A_t dictée par sa stratégie (politique).
3. L'environnement intègre cette action, évolue vers une nouvelle observation, puis retourne à l'agent une nouvelle observation O_{t+1} ainsi qu'une récompense R_{t+1} évaluant la pertinence de la transition effectuée.

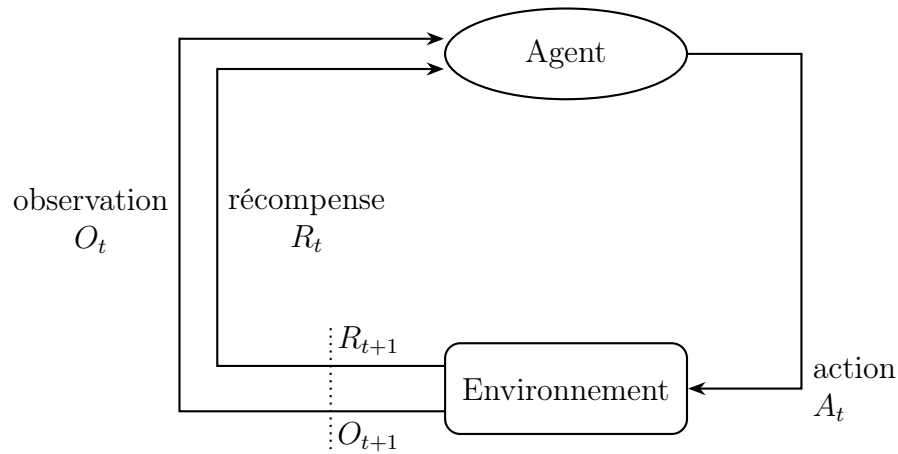


FIGURE 1 – Boucle d’interaction agent-environnement en apprentissage par renforcement

5.1.1 Exemple : Cliff Walking

L’environnement *Cliff Walking* (Farama Foundation, 2025b) (Figure 2) illustre ces mécanismes. L’agent navigue sur une grille pour atteindre une cible en évitant une falaise. Il reçoit une récompense de -1 par pas parcouru, afin d’inciter à emprunter le chemin le plus court, et une pénalité de -100 avec retour au départ s’il tombe. Cet exemple met en évidence un compromis classique : l’agent doit trouver le chemin le plus court vers la cible tout en évitant la falaise, qui entraîne une pénalité sévère.

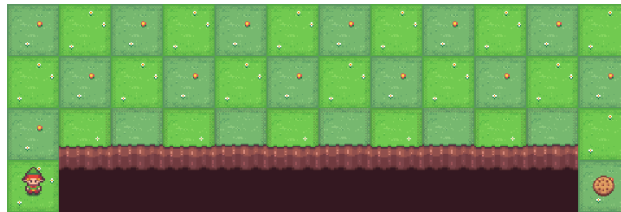


FIGURE 2 – Environnement Cliff Walking.

5.2 Spécificités et Concepts Clés

Le RL se caractérise principalement par l'absence d'expert superviseur (aucun label ne vient indiquer la "bonne" action à prendre, seul le signal de récompense sert de retour), par des signaux de récompense souvent différés, soulignant le problème de l'assignation de crédit¹, et par des données fortement corrélées temporellement, c'est-à-dire non indépendantes et non identiquement distribuées (i.i.d.).

Afin de formaliser ce problème, voici les cinq composantes fondamentales de la boucle d'interaction (Sutton et al., 2018) :

- **État de l'environnement** (S_t) : L'état résume l'ensemble des informations de l'environnement pertinentes à un instant t pour que l'agent puisse déterminer son comportement. L'idéal est qu'il possède la *propriété de Markov* (c'est-à-dire qu'il englobe tout l'historique de manière compacte).
- **Action** (A_t) : La décision exécutée par l'agent dans l'état S_t . Selon le problème, l'espace des actions peut être discret (par exemple : aller en haut, en bas, à gauche, à droite) ou au contraire continu (par exemple : le degré de braquage d'un volant).
- **Récompense** (R_{t+1}) : Le signal scalaire émis par l'environnement qui évalue purement et immédiatement le succès de l'action A_t . L'objectif central du RL est d'optimiser l'accumulation temporelle de ces retours mesurables.
- **Politique** (π) : La stratégie propre à l'agent. Cette fonction détermine quelles actions entreprendre en fonction de chaque état observé. Elle peut être déterministe ($a = \pi(s)$) ou stochastique, c'est-à-dire attribuer une probabilité donnée d'exécuter une action ($P(A_t = a \mid S_t = s)$).
- **Fonction de valeur** (V^π, Q^π) : Mesure la qualité (*value*) d'un état (ou d'une paire état-action), c'est-à-dire l'espérance mathématique des récompenses qui seront cumulées à long terme à partir de cet état, en suivant la politique π .

1. Problème d'assignation de crédit (*Credit Assignment Problem*).

5.3 Processus Décisionnel de Markov (MDP)

L'environnement d'apprentissage dans sa globalité est formellement modélisé par un Processus Décisionnel de Markov (MDP). Ce puissant cadre mathématique repose sur la **propriété de Markov** : l'état actuel S_t suffit à caractériser la dynamique future de l'environnement, indépendamment de tout l'historique passé. La prédiction de l'état suivant dépend ainsi uniquement de l'état présent, indépendamment de tout l'historique de navigation passée ($P(S_{t+1}|S_t) = P(S_{t+1}|H_t)$).

Un MDP se décrit classiquement par le système $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$ et $\mathcal{A}$ représentent respectivement l'ensemble des états que peut prendre l'environnement et l'ensemble des actions appartenant à l'agent.
- $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]$ définit la matrice des probabilités de transition. Elle modélise la nature (déterministe ou probabiliste) des changements d'état résultant d'une action a spécifique.
- $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$ fixe la récompense espérée en transitant par l'action a à partir de l'état s .
- $\gamma \in [0, 1]$ est le facteur d'actualisation (*discount factor*). Il quantifie l'intérêt lié à l'horizon temporel des récompenses : une valeur de γ proche de 0 réduit le temps de vue et favorise les gains très immédiats, tandis qu'une valeur se rapprochant de 1 insiste sur la conception de stratégies à long terme.

L'agent vise in fine à maximiser son **retour cumulé** global G_t , exprimé comme une somme actualisée de ces récompenses :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (1)$$

Pour l'exemple de Cliff Walking, on pourrait imaginer une récompense $\mathcal{R}$ de -1 par étape de déplacement (pour pénaliser le temps perdu) et de +10 en trouvant la sortie, chaque case constituant un état $\mathcal{S}$, et les pas (Gauche, Droite, Haut, Bas) constituant les actions $\mathcal{A}$. Le facteur γ garantit alors que l'agent privilégiera mathématiquement un chemin court et rapide vers la sortie plutôt qu'une errance prolongée.

5.4 Model-Based

Pour classer les méthodes d'apprentissage par renforcement, il est utile de distinguer les approches basées sur un modèle (*model-based*) de celles sans modèle (*model-free*). Les méthodes *model-based* supposent que l'agent possède une connaissance complète de la dynamique de l'environnement, représentée par les fonctions de transition $\mathcal{P}$ et de récompense $\mathcal{R}$.

Grâce à cette connaissance, l'agent peut planifier ses actions en évaluant mentalement leurs conséquences avant même de les exécuter. Ainsi, plutôt que d'apprendre par essais et erreurs, l'agent calcule mathématiquement la stratégie optimale dès le départ, pour autant que la complexité du problème le permette.

5.4.1 Policy Evaluation et Policy Iteration

L'itération de politique (*Policy Iteration*) est un processus qui permet de trouver la politique optimale en alternant deux étapes clés. La première consiste à évaluer la politique actuelle pour connaître sa valeur exacte (*Policy Evaluation*). La seconde étape cherche ensuite à l'améliorer en choisissant systématiquement la meilleure action possible en fonction des résultats de l'évaluation précédente de manière *greedy* (*Policy Improvement*).

Ce processus d'évaluation suivi d'une amélioration est répété itérativement. La convergence vers la politique optimale (π^*) garantit alors systématiquement $V^{\pi^*} \geq V^\pi$ quel que soit l'état initial comparé (Silver, 2015).

Mesurer la performance d'une politique donnée équivaut à quantifier l'espérance mathématique cumulative des récompenses reçues si l'agent respecte continuellement π . Pour ce faire, nous employons une récurrence itérative construite à partir d'une estimation à zéro ($V_0(s) = 0$) :

$$V_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_k(s') \right] \quad (2)$$

Au fil des itérations, les récompenses associées aux actions se propagent

d'état en état (comme l'illustre la Figure 3, où l'influence des états terminaux rayonne vers les cases adjacentes à chaque étape k). Sous réserve d'un nombre infini d'itérations, les calculs convergent et s'affinent.

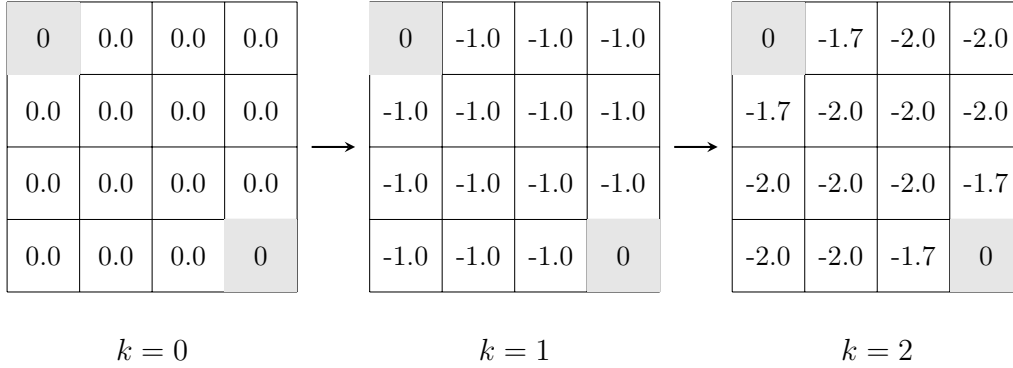


FIGURE 3 – Itérations de l'évaluation de politique dans un monde de grille 4×4 . Les états terminaux sont grisés. Inspiré de Silver (2015).

Bien qu'une évaluation parfaite soit mathématiquement faisable, l'objectif fondamental demeure l'identification rapide d'une stratégie de jeu plus favorable. Dans les faits, un nombre restreint d'itérations k d'évaluation confère une estimation suffisamment robuste pour orienter la décision opportuniste *greedy*.

La mise à jour de la politique s'oriente alors selon le critère de sélection de l'action optimale :

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} q_{\pi}(s, a) \quad (3)$$

Tant que la nouvelle politique générée diffère, ces opérations inter-cycles se relaient. Ce processus prend le nom d'**itération de politique modifiée** (*modified policy iteration*) lorsque l'évaluation s'interrompt pour économiser du temps de calcul dès lors qu'un seuil minime ϵ est atteint entre deux évaluations.

5.4.2 Value Iteration

Contrairement à l'itération de politique qui évalue une stratégie précise à chaque étape, la *Value Iteration* s'attaque à l'optimisation par la fusion des deux étapes précédentes en une seule opération globale. L'algorithme cherche à trouver directement la valeur maximale de chaque état, sans avoir besoin de stocker une politique intermédiaire lors de l'entraînement.

Cette mécanique réside exclusivement dans l'introduction d'un opérateur "maximum" qui condense l'évaluation et l'amélioration :

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V_k(s') \right] \quad (4)$$

Pour chaque état s et à chaque étape, l'agent met à jour ses valeurs en supposant qu'il prendra systématiquement l'action future la plus rémunératrice. En pratique, on arrête les calculs (la phase de convergence) dès que la propagation des valeurs devient marginale d'une itération à l'autre (par exemple lorsque la modification $\max_s |V_{k+1}(s) - V_k(s)| < \epsilon$).

Une fois que les valeurs de l'ensemble V sont stabilisées, l'approche pour définir de quel comportement l'agent doit faire preuve (extraire π^*) se fait intuitivement. Il suffit à l'agent de regarder les états accessibles depuis sa position actuelle et de choisir l'action lui rapportant la valeur la plus élevée :

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V^*(s') \right] \quad (5)$$

5.5 Model-Free

Cependant, l'utilisation de l'apprentissage par renforcement s'applique majoritairement à des environnements réels complexes où les règles du monde sont stochastiques, inconnues, ou trop volumineuses pour être modélisées. Sans la connaissance des formules de probabilité de transition $\mathcal{P}$ ni des fonctions de récompense $\mathcal{R}$, les méthodes *model-based* s'avèrent inapplicables car l'agent ne peut pas se projeter mathématiquement.

Pour surmonter cet obstacle, les méthodes dites *model-free* (sans modèle) effectuent un apprentissage fondé sur l'expérience. Elles s'alimentent exclusivement des données rencontrées au cours de l'interaction de l'agent avec l'environnement pour estimer peu à peu la valeur des actions. Ce changement d'approche correspond à l'idée exprimée par Silver (2015) caractérisant ces processus comme une étape de véritable apprentissage par expérience (*learning*) plutôt que de simple planification en amont (*planning*).

5.5.1 Monte-Carlo

L'approche de *Monte-Carlo* (MC) (Sutton et al., 2018) s'appuie sur une idée très intuitive en ce qui concerne l'apprentissage basé sur l'expérience : la valeur d'un état correspond simplement à la moyenne de la somme des récompenses accumulées lors des passages par cet état. À l'inverse des méthodes *Temporal Difference* (*TD*) (voir Section 5.5.2), qui mettent à jour les estimations en s'appuyant sur d'autres estimations de valeurs (créant ainsi une chaîne de prédictions imbriquées), Monte-Carlo se fie uniquement aux résultats complets observés directement sans dépendre d'approximations intermédiaires.

Le fonctionnement de base des méthodes de Monte-Carlo repose sur trois prérequis. Le premier est la nécessité de générer un apprentissage par une séquence d'expériences (états, actions, et récompenses) accumulée par l'agent au cours de ses essais successifs. Par corrélation, l'environnement étudié doit obligatoirement être de nature "épisodique". C'est-à-dire qu'une interaction doit toujours avoir une fin déterminée, au contraire de processus cycliques tournant infiniment. En effet, tout le principe repose sur le fait de pouvoir calculer le retour accumulé et total de la session, ce qui requiert d'atteindre la fin de l'épisode (au *game over* d'une partie d'échecs par exemple) pour être possible. Naturellement, ces données servent une estimation qui, répétée pour chaque nouvel essai, s'affine statistiquement au fil de l'augmentation du nombre de lancers.

Pour des raisons d'efficacité, l'algorithme ne garde pas en mémoire l'intégralité des résultats passés pour calculer sa moyenne de zéro à chaque fois. Il utilise plutôt une méthode de mise à jour incrémentale. À la fin de chaque épisode, il passe en revue les états qu'il a visités et corrige directement sa

dernière estimation :

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t)) \quad (6)$$

Ici, la différence $(G_t - V(S_t))$ exprime très exactement l'erreur mesurée entre le résultat du retour nouvellement calculé en finissant l'épisode (G_t) et l'estimation jusqu'alors théorisée pour l'état en passe d'être mis à jour ($V(S_t)$). Cette erreur est tempérée par un coefficient α , le taux d'apprentissage (*learning rate*), ce qui évite un bouleversement complet des connaissances en cas de coup de chance et lisse l'apprentissage.

Au cours d'un épisode, un état précis peut faire l'objet de visites répétées (comme l'agent passant par une case du labyrinthe plusieurs fois de suite). Ce détail force l'approche de l'algorithme à privilégier deux méthodes de traitement distinctes. Avec le premier traitement (*First-Visit*), l'algorithme relève seulement et ajuste la valeur basée sur ce qui s'est déroulé après avoir touché l'état la toute première fois au cours d'une occurrence. La méthode alternative (*Every-Visit*) reconsidère l'évaluation de l'état pour chacune de ses apparitions.

Si cette méthode présente l'avantage d'être très robuste (ses estimations sont toujours basées sur des résultats réels et ne sont donc pas perturbées par de potentielles prédictions erronées sur d'autres états), elle souffre cependant de défauts majeurs dans son rythme d'apprentissage. En effet, l'agent doit systématiquement attendre la fin définitive de l'épisode pour mettre à jour ses connaissances, ce qui rend la progression particulièrement lente selon l'environnement. De plus, cela crée un problème critique de forte variance : un coup de chance exceptionnel ou un désastre va impacter le score final de tout l'épisode. Cela peut conduire à des mises à jour très versatiles et irrégulières, rendant l'apprentissage instable et difficile à maîtriser.

5.5.2 Temporal-Difference (TD)

Les méthodes des différences temporelles (TD) combinent les forces des méthodes de Monte-Carlo et de la programmation dynamique. À l'instar de Monte-Carlo, elles apprennent directement à partir de l'expérience brute,

sans nécessiter de modèle de transition ($\mathcal{P}$). Cependant, à l'instar de la programmation dynamique, elles s'affranchissent de l'attente de la fin de l'épisode en utilisant le *bootstrapping*² (Sutton et al., 2018).

Pour illustrer cette nuance, Silver (2015) propose l'analogie du trajet en voiture. Imaginons un conducteur estimant arriver chez lui à 18h30. À 18h15, il se retrouve bloqué dans un embouteillage. Une approche Monte-Carlo obligerait le conducteur à attendre d'être effectivement garé chez lui pour constater son retard et ajuster ses prévisions futures. À l'inverse, l'approche TD permet au conducteur de *bootstrapper* : dès qu'il voit le bouchon, il actualise instantanément son estimation ("le trajet prendra finalement 50 minutes") sans avoir besoin de terminer sa route.

Mathématiquement, le mécanisme le plus simple, le TD(0), remplace le gain réel G_t par une **cible estimée** (*TD target*) composée de la récompense immédiate et de la valeur de l'état suivant :

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))}_{\text{TD target}} \quad (7)$$

La différence entre cette cible et l'estimation actuelle est nommée **erreur TD** ($\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$). Contrairement à Monte-Carlo, le TD(0) peut fonctionner dans des environnements continus ou cycliques car il apprend à chaque pas.

Par nature, le TD(0) ne regarde qu'un seul pas dans le futur avant de consulter son estimation. On peut toutefois généraliser ce concept en enregistrant n étapes de récompenses réelles avant de *bootstrapper* sur l'état S_{t+n} . On obtient alors le **retour à n pas** (*n-step return*) :

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}) \quad (8)$$

Il convient de noter que le terme de *bootstrapping* $\gamma^n V(S_{t+n})$ n'est valide que si l'épisode n'est pas terminé avant l'étape $t + n$. Si l'épisode se termine à

2. Principe consistant à mettre à jour l'estimation d'un état en se basant sur l'estimation de l'état suivant, plutôt que sur le retour final réel. (Sutton et al., 2018)

$t + k$ avec $k < n$, ce terme disparaît et le retour se réduit à la somme réelle des récompenses obtenues, ce qui équivaut à l'estimateur de Monte-Carlo.

Si $n = 1$, l'approche équivaut au TD(0). Si $n \rightarrow \infty$, elle rejoint la méthode de Monte-Carlo. Le choix de n devient alors un curseur permettant de modifier l'horizon de prédiction de l'agent.

Cependant, plutôt que de choisir arbitrairement ce nombre de pas ($n = 1, 2, \dots, \infty$), l'algorithme $TD(\lambda)$ propose une unification de tous ces horizons en les pondérant simultanément. Ce mélange est réalisé via une moyenne pondérée par un paramètre $\lambda \in [0, 1]$, définissant ainsi le **λ -return** (G_t^λ) :

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} \quad (9)$$

Bien que le G_t^λ offre une cible robuste, il souffre du même défaut que Monte-Carlo : il regarde vers le futur et nécessite d'attendre la fin de l'épisode pour être calculé (approche dite *Forward View*). Pour rendre l'apprentissage possible en temps réel, on adopte une approche *Backward View* s'appuyant sur les **traces d'éligibilité** $E_t(s)$ (Silver, 2015).

L'équivalence mathématique entre ces deux visions permet de ne plus utiliser directement le λ -return dans la mise à jour, mais de propager l'erreur TD locale (δ_t) vers le passé. Chaque état visité laisse une "empreinte" qui s'estompe avec le temps selon le facteur λ ³. La mise à jour de la trace d'éligibilité pour chaque état s à chaque pas de temps t s'écrit ainsi :

$$E_t(s) = \gamma \lambda E_{t-1}(s) + 1(S_t = s) \quad (10)$$

Ainsi, à chaque pas de temps, la fonction de valeur de **tous les états** est ajustée proportionnellement à leur responsabilité dans l'erreur courante :

$$V(s) \leftarrow V(s) + \alpha \delta_t E_t(s) \quad (11)$$

3. Le paramètre λ contrôle la persistance de la trace : une valeur proche de 0 restreint la mise à jour aux états les plus immédiats (approche "myope"), tandis qu'une valeur proche de 1 maintient l'empreinte des états anciens plus longtemps.

Cette mécanique est particulièrement efficace : dans un jeu vidéo, si un agent saute sur plusieurs plateformes avant de tomber dans un piège, le $TD(\lambda)$ permettra de baisser instantanément la valeur de toutes les plateformes de la séquence (beaucoup pour la dernière, un peu moins pour les précédentes) au prorata de leur responsabilité dans l'issue finale.

5.6 Apprentissage en environnement inconnu

Le passage de l'évaluation à l'optimisation consiste à améliorer la politique de l'agent sans connaissance préalable de la dynamique de l'environnement. Dans ce contexte *model-free*, l'utilisation de la fonction de valeur d'état $V(s)$ s'avère insuffisante : pour choisir l'action optimale, l'agent aurait besoin de connaître les probabilités de transition $\mathcal{P}$. Par conséquent, les algorithmes de contrôle se basent sur la fonction action-valeur $Q(s, a)$, permettant d'identifier directement l'action maximisant le gain attendu (Silver, 2015).

5.6.1 On-Policy vs Off-Policy

Deux paradigmes fondamentaux s'opposent dans l'apprentissage sans modèle :

- **On-Policy** : L'agent évalue et améliore directement la politique qu'il exécute. Les expériences utilisées pour la mise à jour reflètent ses propres décisions, y compris ses stratégies d'exploration.
- **Off-Policy** : L'agent optimise une politique cible π en s'appuyant sur des données générées par une politique de comportement distincte μ . Cette séparation permet d'utiliser des expériences collectées par d'autres politiques pour l'apprentissage (Sutton et al., 2018).

5.6.2 Le dilemme Exploration-Exploitation

Pour garantir la convergence vers une solution idéale, l'agent ne peut se contenter d'être *greedy* dès le départ, au risque de rester piégé dans un optimum local. Il doit arbitrer entre l'exploitation de ses connaissances actuelles (issues de l'évaluation) et l'exploration de nouvelles stratégies. La méthode

la plus répandue est la stratégie **ϵ -greedy** :

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{m} & \text{si } a = \operatorname{argmax}_{a \in \mathcal{A}} Q(s, a) \\ \frac{\epsilon}{m} & \text{sinon} \end{cases} \quad (12)$$

En pratique, on utilise une fonction de décroissance de ϵ (*scheduler*) pour réduire progressivement sa valeur au fil du temps. Cette décroissance permet de satisfaire les conditions **GLIE**⁴ : l'agent explore massivement au début, puis stabilise son comportement vers une politique déterministe optimale une fois l'espace état-action suffisamment parcouru. Ces conditions sont importantes car elles constituent une garantie théorique de convergence : à condition que chaque paire état-action soit visitée un nombre infini de fois et que la politique devienne progressivement gloutonne, les algorithmes de contrôle comme SARSA sont assurés de converger vers la politique optimale π^* (Singh et al., 2000). Cependant, un *epsilon* qui descendrait trop vite à 0 créerait un manque d'exploration, empêchant l'agent de découvrir l'intégralité des récompenses et biaisant l'optimisation.

5.6.3 SARSA

L'algorithme **SARSA** (Sutton et al., 2018) est une méthode d'optimisation **On-Policy**. Il met à jour la fonction Q en utilisant la récompense immédiate et l'évaluation de l'action suivante (A') effectivement choisie par la politique actuelle de l'agent. Son nom illustre d'ailleurs la séquence (S, A, R, S', A') utilisée pour la mise à jour :

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A)) \quad (13)$$

Parce qu'il inclut l'action réelle A' de l'agent (et donc les erreurs potentielles liées à l'exploration) dans son calcul, SARSA réalise une optimisation "prudente". Il évalue les zones de l'environnement en tenant compte du fait qu'un mouvement aléatoire dû à ϵ pourrait entraîner une chute brutale de la récompense (Silver, 2015).

4. *Greedy in the Limit with Infinite Exploration*

5.6.4 Q-Learning

À l'inverse, le Q-Learning (Watkins et al., 1992) est une méthode d'optimisation **Off-Policy**. Il s'affranchit de l'action réellement choisie par l'agent pour la mise à jour en utilisant l'opérateur *maximum*. Il optimise directement la valeur de la politique optimale, indépendamment du comportement exploratoire :

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma \max_{a'} Q(S', a') - Q(S, A)) \quad (14)$$

Ici, par rapport à SARSA, la mise à jour se base sur l'évaluation de la meilleure action possible (max), sans tenir compte de l'action A_{t+1} réellement effectuée par l'agent. Cette approche permet ainsi de converger vers la politique optimale via l'équation d'optimalité de Bellman, même si l'agent continue d'explorer activement son environnement.

5.7 Du RL Tabulaire au Deep RL

Les méthodes tabulaires présentées précédemment, bien que robustes, se heurtent à la *malédiction de la dimensionnalité*. Dans des environnements complexes (qu'il s'agisse de flux de pixels ou de vecteurs d'états continus), le nombre de paires (s, a) devient trop vaste pour être stocké ou exploré exhaustivement.

Pour surmonter cette limite, l'**approximation de fonction** est une idée centrale. L'objectif n'est plus de maintenir un tableau de valeurs exactes, mais d'apprendre une fonction paramétrée $\hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$ ⁵, capable de généraliser les connaissances acquises vers des états jamais rencontrés.

5.7.1 Le défi de la stabilité

Le *Deep Reinforcement Learning* (DRL) exploite la puissance des réseaux de neurones profonds comme approximateurs de fonction universels. Cependant, l'intégration de ces réseaux dans une boucle d'optimisation *model-free*

5. $\mathbf{w}$ le vecteur de paramètres (poids) du réseau de neurones.

est intrinsèquement instable. Cette instabilité est théorisée par la **Triade Mortelle** (*Deadly Triad*) (Sutton et al., 2018), qui survient lorsque trois éléments sont réunis :

- **L'approximation de fonction** (utilisation de réseaux de neurones) ;
- **Le *bootstrapping*** (mise à jour d'estimations basées sur d'autres estimations) ;
- **L'apprentissage Off-Policy** (utilisation de données issues de politiques antérieures).

Sous ces conditions, la convergence des algorithmes est menacée par une forte corrélation temporelle des données et une cible de calcul mouvante (le réseau tente de rattraper une valeur qu'il modifie lui-même).

5.7.2 Deep Q-Network

L'algorithme **Deep Q-Network (DQN)** (Mnih et al., 2015) a marqué l'avènement du DRL en proposant deux solutions techniques majeures pour briser cette triade et stabiliser l'optimisation :

1. **Experience Replay** : Au lieu d'apprendre de manière séquentielle sur la donnée immédiate, l'agent stocke ses transitions (s, a, r, s') dans une mémoire de rejeu (*Replay Buffer*). Il échantillonne ensuite aléatoirement des "mini-batches" d'expériences passées pour s'entraîner. Cela casse la corrélation temporelle des trajectoires et rend l'apprentissage plus stable en se rapprochant des conditions *i.i.d.* (indépendantes et identiquement distribuées) de l'apprentissage supervisé.
2. **Target Network** : Pour éviter que la cible de mise à jour ne change à chaque itération, DQN utilise deux réseaux distincts. Un réseau *online* (paramétré par $\mathbf{w}$) qui est optimisé activement, et un réseau "cible" (*target*, paramétré par $\mathbf{w}^-$) dont les poids sont gelés et périodiquement synchronisés avec le premier.

L'apprentissage s'effectue en minimisant l'erreur quadratique moyenne entre la prédiction du réseau online et la cible fournie par le réseau target via une étape de descente de gradient. La fonction de perte (*loss*) s'écrit formellement :

$$L(\mathbf{w}) = \mathbb{E}_{(s,a,r,s') \sim \text{Replay}} \left[\left(r + \gamma \max_{a'} \hat{q}(s', a', \mathbf{w}^-) - \hat{q}(s, a, \mathbf{w}) \right)^2 \right] \quad (15)$$

Cette architecture permet de converger efficacement dans des environnements à large espace d'états (comme le traitement d'images/pixels de jeux vidéo). Il est toutefois fondamental de retenir que l'algorithme DQN est intrinsèquement restreint aux **espaces d'actions discrets**. En effet, le calcul de la cible repose sur l'évaluation de la meilleure action future via l'opérateur $\max_{a'} \hat{q}(s', a')$. Si l'espace des actions était continu, rechercher algorithmiquement le maximum global d'une fonction neuronale hautement non linéaire à chaque itération et pour chaque transition représenterait un coût informatique prohibitif.

Pour pallier cette limitation, DDPG (*Deep Deterministic Policy Gradient*) (Lillicrap et al., 2015) étend les principes de DQN aux espaces continus en apprenant simultanément une politique déterministe (l'*Acteur*) et une fonction de valeur (le *Critique*), supprimant ainsi la nécessité de l'opérateur max. DDPG constitue la fondation architecturale directe de SAC, qui en corrige les principaux problèmes de stabilité et d'exploration, comme détaillé dans la section suivante.

5.8 Méthodes Actor-Critic

Pour pallier l'incompatibilité de l'algorithme DQN avec les espaces d'actions continus (comme les couples moteurs de la robotique ou le pilotage de véhicules), une approche radicalement différente est exploitée : le **Gradient de Politique** (*Policy Gradient*).

L'idée est de s'affranchir de la recherche de la valeur maximale des actions pour dicter le comportement. On y optimise plutôt directement les paramètres θ d'une politique stochastique $\pi_\theta(a|s)$ par montée de gradient, dans l'optique de maximiser le retour algorithmique attendu $J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$.

5.8.1 Architecture Actor-Critic

Les méthodes modernes de l'état de l'art fusionnent l'apprentissage de la fonction de valeur et l'optimisation directe de la politique dans une architecture hybride nommée **Actor-Critic** (Sutton et al., 2018) :

- **L'Acteur** ($\pi_\theta(a|s)$) : Un réseau de neurones constituant la politique. Il décide quelle action entreprendre.
- **Le Critique** ($V_w(s)$ ou $Q_w(s, a)$) : Un second réseau agissant en tant qu'évaluateur. Il calcule l'**avantage** estimé d'une action, défini comme $\hat{A}(s, a) = Q(s, a) - V(s)$, qui mesure dans quelle mesure l'action choisie est meilleure ou moins bonne que le comportement moyen attendu de la politique en cet état. Soustraire $V(s)$ joue ici le rôle de référence (*baseline*) : ce résultat fondamental, établi par Williams (1992), démontre qu'une référence ne biaise pas l'estimation du gradient de politique tout en réduisant significativement sa variance. Sans cette soustraction, le signal de mise à jour de l'Acteur serait extrêmement bruité, rendant l'apprentissage instable. Le Critique offre ainsi un signal qualitatif clair et à faible variance pour guider la mise à jour de l'Acteur.

L'avantage majeur de ce duo est qu'il paramètre intrinsèquement très bien la nature des actions (discrètes ou continues). Pour un problème *discret*, la couche de sortie de l'Acteur applique une fonction *Softmax* définissant la probabilité de tirer chaque action de la liste. Pour un espace *continu*, l'Acteur va générer non pas des actions brutes, mais les paramètres statistiques d'une loi de distribution continue (typiquement la moyenne μ et l'écart-type σ d'une variable gaussienne), au sein de laquelle l'action réelle sera piochée aléatoirement.

5.8.2 Proximal Policy Optimization (PPO)

L'algorithme **PPO** (Schulman ; Wolski et al., 2017) est devenu un standard industriel du RL pour son équilibre entre simplicité d'implémentation et grande robustesse empirique.

Estimation de l'avantage via GAE. PPO est fréquemment combiné avec l'estimateur **GAE** (*Generalized Advantage Estimation*) (Schulman ; Moritz et al., 2018), qui généralise le retour à n pas pour l'estimation de l'avantage. Plutôt que de fixer un horizon n arbitraire, GAE calcule une moyenne pondérée exponentiellement de toutes les erreurs TD successives, contrôlée par un paramètre $\lambda_{\text{GAE}} \in [0, 1]$:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda_{\text{GAE}})} = \sum_{l=0}^{\infty} (\gamma \lambda_{\text{GAE}})^l \delta_{t+l} \quad (16)$$

où $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ est l'erreur TD locale. λ_{GAE} joue le même rôle que λ dans TD(λ) : une valeur proche de 0 produit une estimation à faible variance mais potentiellement biaisée (horizon court), tandis qu'une valeur proche de 1 réduit le biais au prix d'une variance plus élevée (horizon long). Ce paramètre sera directement retrouvé lors de la configuration expérimentale de PPO (section 8.2.3).

Fonction objectif écrêtée (*Clipped Surrogate Objective*). Un problème fondamental du gradient de politique est que des mises à jour trop importantes peuvent détruire une politique fonctionnelle de manière irréversible. PPO sécurise l'apprentissage en introduisant une fonction objectif **écrêtée** qui limite strictement l'amplitude des modifications entre deux mises à jour de la politique :

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (17)$$

Les trois termes de cette expression (que l'on cherche à maximiser) s'interprètent comme suit :

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ est le ratio de probabilité. Il quantifie si l'action a_t est plus ($r_t > 1$) ou moins ($r_t < 1$) probable sous la nouvelle politique que sous l'ancienne.
- $\hat{A}_t$ est l'avantage estimé par le Critique, indiquant si l'action s'est révélée meilleure ou pire que l'espérance de l'état.
- La fonction clip borne ce ratio dans l'intervalle $[1 - \varepsilon, 1 + \varepsilon]$ (typiquement $\varepsilon = 0.2$), supprimant toute incitation à s'en écarter davantage.

L'opérateur min garantit une politique conservatrice dans les deux sens : si une action a été bonne ($\hat{A}_t > 0$), on cherche à augmenter sa probabilité, mais uniquement dans la limite autorisée ; si elle a été mauvaise ($\hat{A}_t < 0$), on la pénalise, là encore dans la même fenêtre bornée.

Notion de rollout et efficacité des données. En pratique, PPO collecte un **rollout** — une séquence de n transitions consécutives — avant d’effectuer ses mises à jour. Ce batch de données est exploité sur plusieurs passes d’optimisation (n_{epochs}), découpé en mini-batches de taille b . Pour un rollout de 2048 steps, des mini-batches de 256 et 20 époques, PPO effectue ainsi $\lfloor 2048/256 \rfloor \times 20 = 160$ mises à jour du réseau par rollout. Ce mécanisme compense partiellement la *sample efficiency* structurellement limitée des méthodes *on-policy* ; néanmoins, le batch est intégralement renouvelé après chaque cycle, les données devenant non représentatives dès que la politique est modifiée (Schulman ; Wolski et al., 2017).

5.8.3 Soft Actor-Critic (SAC)

SAC (Haarnoja ; Zhou ; Abbeel et al., 2018) étend le cadre actor-critic en intégrant l’**entropie** de la politique à la fonction objectif. L’agent cherche à maximiser simultanément les récompenses accumulées et l’entropie de sa politique, afin de rester exploratoire et d’éviter une convergence prématurée vers une stratégie sous-optimale :

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \pi} \left[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right] \quad (18)$$

où $\mathcal{H}(\pi(\cdot | s_t)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s_t)]$ est l’entropie de Shannon de la politique en l’état s_t , et $\alpha > 0$ est un coefficient de température pondérant l’importance relative de l’entropie.

Ajustement automatique du coefficient d’entropie. Le coefficient α peut être fixé manuellement ou ajusté automatiquement. Haarnoja ; Zhou ; Hartikainen et al. (2019) proposent d’optimiser α à chaque step de gradient pour maintenir une entropie cible $\mathcal{H}^*$, définie par défaut comme l’opposé de la dimension de l’espace d’action :

$$\mathcal{H}^* = -\dim(\mathcal{A}) \quad (19)$$

Ce choix correspond approximativement à l'entropie d'une distribution uniforme sur un espace d'action normalisé, garantissant une exploration suffisante sans sur-contraindre la politique.

Reparameterization trick. SAC exploite le **reparameterization trick** pour rendre son optimisation différentiable. Échantillonner directement une action aléatoire $a \sim \pi_\theta(\cdot|s)$ est une opération non différentiable qui bloque la rétropropagation du gradient.

State-Dependent Exploration (SDE). Par défaut, SAC rééchantillonne un bruit gaussien indépendant à chaque pas de temps, ce qui peut produire une exploration sans cohérence temporelle. Raffin et al. (2021) proposent une alternative, la **State-Dependent Exploration (SDE)** : plutôt que de tirer un nouveau bruit à chaque step, SDE maintient un vecteur de bruit ε fixe pendant plusieurs steps consécutifs et apprend un mapping $f(s, \varepsilon)$ dépendant de l'état courant. L'exploration devient ainsi cohérente dans le temps — l'agent explore des trajectoires entières plutôt que des actions isolées — et structurée par l'état, ce qui est particulièrement adapté aux tâches de contrôle continu nécessitant des séquences d'actions coordonnées.

Double réseau critique. Pour contenir le biais de surestimation des valeurs Q (*overestimation bias*) — problème structurel des méthodes actor-critic identifié par Fujimoto et al. (2018) — SAC adopte le principe du **double réseau critique** hérité de TD3 (*Twin Delayed DDPG*), l'extension de DDPG proposée par Fujimoto et al. (2018) pour corriger ce biais :

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s'), \quad \tilde{a}' \sim \pi_\theta(\cdot|s') \quad (20)$$

En prenant le minimum, on pénalise les états sur-estimés plutôt que de les amplifier, ce qui stabilise l'apprentissage du critique et, par ricochet, celui de l'acteur.

Soft target update. Contrairement à DQN qui synchronise périodiquement son réseau cible par copie intégrale des poids, SAC adopte une mise à jour **douce** (*soft update*) à chaque step de gradient pour chacun de ses deux réseaux cibles :

$$\phi_{\text{cible}} \leftarrow \tau \phi + (1 - \tau) \phi_{\text{cible}} \quad (21)$$

où $\tau \ll 1$ (valeur standard : 0.005 (Haarnoja ; Zhou ; Abbeel et al., 2018)) est le taux d’interpolation. Cette approche produit une cible évoluant lentement et continûment, offrant un signal d’apprentissage plus stable que les mises à jour discrètes de DQN. C’est l’ensemble de cette architecture — deux critiques, deux réseaux cibles, un acteur, et le mécanisme d’entropie adaptative — qui explique généralement le coût computationnel structurellement plus élevé de SAC par rapport à des algorithmes n’entretenant qu’un seul réseau critique.

5.9 Synthèse et positionnement des algorithmes étudiés

Les trois algorithmes retenus pour cette étude ont été délibérément choisis pour couvrir des paradigmes d’apprentissage complémentaires. Le tableau 1 résume leurs caractéristiques fondamentales selon les axes de comparaison qui structureront l’analyse expérimentale.

TABLE 1 – Comparaison des algorithmes DQN, PPO et SAC selon leurs caractéristiques fondamentales.

Critère	DQN	PPO	SAC
Paradigme	Off-Policy	On-Policy	Off-Policy
Espace d’actions	Discret	Discret / Continu	Continu
Replay Buffer	✓	×	✓
<i>Sample Efficiency</i>	Moyenne	Faible	Élevée
Stabilité	Moyenne	Élevée	Élevée
Exploration	ϵ -greedy	Stochastique (distribution apprise)	Maximisation d’entropie
Référence fondatrice	Mnih et al. (2015)	Schulman ; Wolski et al. (2017)	Haarnoja ; Zhou ; Abbeel et al. (2018)

Note — L’exploration de PPO est intrinsèquement stochastique : la politique paramétrise une distribution de probabilités (catégorielle en discret, gaussienne en

continu) depuis laquelle les actions sont échantillonnées à chaque step. Contrairement à DQN, aucun mécanisme ε -greedy n'est nécessaire. Le paramètre ε de PPO désigne le seuil d'écrtage du Clipped Surrogate Objective, sans lien avec la stratégie d'exploration.

Note sur la Sample Efficiency — Ce classement reflète le nombre d'interactions avec l'environnement nécessaires pour atteindre une performance donnée. DQN, bien qu'*off-policy* et doté d'une mémoire de rejeu, reste pénalisé par l'instabilité de son opérateur max et l'absence de mécanisme correctif contre la surestimation des valeurs Q, ce qui limite le gain réel tiré de la réutilisation des transitions passées. PPO, à l'inverse, rejette l'intégralité de son batch de données après chaque cycle de mises à jour (voir 5.8.2), ce qui en fait structurellement la méthode la moins *sample-efficient* des trois malgré ses multiples époques d'optimisation par rollout. SAC combine mémoire de rejeu et double critique, ce qui explique son efficacité supérieure.

Cette diversité de paradigmes permet une comparaison expérimentale riche : DQN sert de référence tabulaire profonde, PPO représente l'approche On-Policy de l'état de l'art, et SAC incarne l'efficacité Off-Policy avec exploration intrinsèque. Leur confrontation sur des environnements Gymnasium standardisés, puis sur un environnement complexe avec *reward shaping*, constituera le cœur de l'analyse empirique de ce mémoire.

6 Problématique

L'apprentissage par renforcement (RL) est un domaine riche mais dont la mise en pratique soulève de nombreux défis techniques. Ce mémoire a pour objectif d'explorer concrètement ce domaine à travers deux grands axes.

Le premier axe concerne le problème d'optimisation et de sélection des modèles. Face à un problème donné, plusieurs décisions cruciales s'imposent : comment choisir d'abord le bon algorithme (DQN, PPO, SAC, etc.) en fonction des spécificités de l'environnement (comme des espaces d'actions discrets ou continus) ? Ensuite, comment relever le défi de l'optimisation des hyperparamètres, étape indispensable pour faire converger ces modèles au sein d'environnement ?

Le second axe se concentre sur l'alignement du comportement de l'agent avec les attentes du concepteur à travers le *reward shaping* (l'ingénierie de la récompense). Dans un environnement complexe, définir la récompense s'avère

extrêmement délicat. L'enjeu est ici de comprendre les problématiques liées à cette conception : comment construire un signal qui guide efficacement l'apprentissage, tout en évitant les comportements opportunistes inattendus (*reward hacking*) liés à une fonction mal spécifiée.

Ainsi, la problématique centrale de ce travail est de comprendre la méthodologie pratique du RL : de la sélection et du réglage des algorithmes sur des cas d'étude standard, jusqu'aux défis complexes de la conception des récompenses (*reward shaping*) nécessaires pour résoudre des environnements avancés.

7 Outils utilisés

Dans ce mémoire, plusieurs outils et bibliothèques ont été mobilisés pour conduire les expériences d'apprentissage par renforcement.

7.1 Gymnasium

Gymnasium (anciennement OpenAI Gym, renommé et maintenu par la Farama Foundation depuis 2022) est une bibliothèque de référence pour la création et l'utilisation d'environnements d'apprentissage par renforcement. Elle expose une interface standardisée permettant d'interagir de manière uniforme avec des environnements très variés, ce qui facilite le développement et la comparaison d'algorithmes de RL. Gymnasium propose un large catalogue d'environnements, allant des jeux classiques aux tâches de contrôle continu, ce qui en fait un socle incontournable pour tester et évaluer les performances des agents. (Farama Foundation, 2025a)

7.2 Stable Baselines3

Stable Baselines3 est une bibliothèque de référence pour l'implémentation d'algorithmes d'apprentissage par renforcement reposant sur PyTorch. Elle fournit des implémentations fiables et validées d'algorithmes de référence tels

que DQN, PPO ou SAC. Conçue pour allier simplicité d'utilisation et hautes performances, elle constitue un choix naturel aussi bien pour la recherche que pour des applications pratiques en RL. (RAFFIN et al., 2026)

7.3 Optuna

Optuna est une bibliothèque d'optimisation d'hyperparamètres permettant d'identifier efficacement les configurations les plus performantes pour un modèle donné. Contrairement aux approches exhaustives de type *grid search*, qui parcourent l'espace des hyperparamètres de manière séquentielle et aveugle, Optuna s'appuie sur un algorithme bayésien (*Tree-structured Parzen Estimator*) pour orienter intelligemment l'exploration vers les régions prometteuses de l'espace de configuration.

Par ailleurs, Optuna intègre un mécanisme d'élagage (*pruning*) qui permet d'interrompre prématurément les essais peu prometteurs, sur la base d'une évaluation continue des performances en cours d'entraînement. Cette double approche — échantillonnage bayésien guidé et élagage précoce — s'avère particulièrement précieuse en apprentissage par renforcement, où l'entraînement des agents est à la fois coûteux en ressources de calcul et très sensible aux choix d'hyperparamètres. (Akiba et al., 2019)

7.4 Weights & Biases

Weights & Biases (WandB) est une plateforme de suivi et de visualisation des expériences d'apprentissage automatique. Elle permet d'enregistrer les métriques d'entraînement, de visualiser les courbes d'apprentissage et de comparer les performances de différents modèles et configurations. WandB facilite par ailleurs la collaboration en centralisant l'ensemble des résultats expérimentaux et en offrant des outils de partage adaptés au travail en équipe. (Weight & Biases, 2017)

7.5 RLGym

RLGym est une bibliothèque de simulation d’environnements pour l’apprentissage par renforcement, initialement conçue pour le jeu Rocket League, mais désormais indépendante de celui-ci. Elle s’appuie sur les abstractions de Gymnasium et propose des outils spécifiques à la simulation de physique de jeu pour l’entraînement d’agents. (RLGym, 2020b)

7.5.1 RocketSim

RocketSim est une bibliothèque de simulation physique de Rocket League, optimisée pour les scénarios d’entraînement intensif. Son principal atout réside dans la prise en charge du *headless training* — un entraînement sans interface graphique — qui supprime toute charge de rendu visuel et accélère considérablement les cycles d’expérimentation. (ZealanL, 2022)

7.5.2 RocketSimVis

RocketSimVis est une bibliothèque de visualisation complémentaire à RocketSim. Elle permet de rejouer et d’inspecter visuellement les parties simulées, ce qui s’avère précieux pour analyser les comportements des agents et appréhender les dynamiques de l’environnement. (ZealanL, 2024)

7.5.3 RLBot

RLBot est un framework open-source permettant de développer et de déployer des agents autonomes personnalisés au sein de Rocket League. Il fournit une interface pour interagir avec le moteur du jeu, facilitant ainsi la création d’adversaires aux comportements complexes. Son fonctionnement est strictement restreint aux sessions hors-ligne ; cette limitation technique garantit l’absence d’interférence avec les fonctionnalités en ligne et assure une utilisation conforme aux conditions d’utilisation du jeu, éliminant tout risque de bannissement. (RLBot, 2018)

7.5.4 RL Gym-PPO

`rlgym-ppo` est une implémentation vectorisée de l'algorithme PPO, développée spécifiquement pour s'interfacer avec RL Gym et RocketSim. Contrairement à des frameworks plus généralistes comme Stable Baselines3 (Section 7.2), conçus pour s'interfacer avec des environnements Gymnasium standards, `rlgym-ppo` est spécifiquement optimisé pour la collecte massive et parallélisée de transitions à travers un grand nombre d'instances simultanées de RocketSim, ce qui le rend particulièrement adapté à des besoins d'entraînement à grande échelle (de l'ordre du milliard de pas). Cette implémentation simplifie par ailleurs le suivi des métriques d'entraînement via son intégration native avec Weights & Biases (Section 7.4), ainsi que la gestion des checkpoints pour la sauvegarde et la reprise de l'entraînement. (Allen, 2023)

7.5.5 RL Gym-SAC

`rlgym-sac` est une adaptation de `rlgym-ppo` reprenant son infrastructure de collecte vectorisée à l'algorithme SAC. Il reprend l'essentiel des composants d'ingénierie de `rlgym-ppo` — parallélisation des instances RocketSim, intégration Weights & Biases, gestion des checkpoints — en substituant l'algorithme d'apprentissage sous-jacent. Ce projet, moins mature et moins largement adopté par la communauté que `rlgym-ppo`, présente plusieurs limites d'implémentation, détaillées en Section 9.6.4. (McSwain, 2026)

8 Phase I - Validation méthodologique

Avant d'aborder des environnements plus complexes, cette phase préliminaire vise à maîtriser les outils et pratiques du RL moderne : optimisation systématique des hyperparamètres, monitoring de la convergence et élagage automatique des configurations non prometteuses. L'environnement *Lunar Lander* de la suite OpenAI Gymnasium constitue un terrain d'entraînement standard et bien documenté pour valider ces pratiques.

8.1 Environnement et algorithmes sélectionnés

LunarLander (Farama Foundation, 2025c) est un problème de contrôle classique dans lequel un agent doit poser un module lunaire entre deux drapeaux situés en $(0,0)$ en gérant sa poussée et son orientation. Deux variantes sont disponibles :

- **LunarLander (discret)** : l'agent choisit à chaque pas de temps parmi quatre actions discrètes (ne rien faire, allumer le propulseur gauche, principal ou droit).
- **LunarLanderContinuous (continu)** : l'espace d'action est continu et bidimensionnel, correspondant à l'intensité de chaque propulseur.

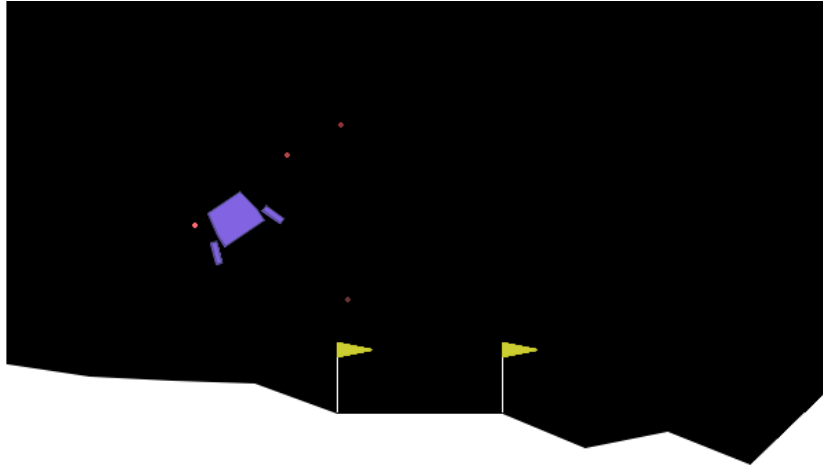


FIGURE 4 – Aperçu de l'environnement LunarLander

L'espace d'observation est identique dans les deux cas : un vecteur de huit variables continues décrivant la position, la vitesse, l'angle, la vitesse angulaire et le contact des deux pattes avec le sol.

La fonction de récompense pénalise la consommation de carburant et la distance à la zone d'atterrissage, et accorde un bonus lors du contact des pattes avec le sol ainsi qu'une récompense de $+100$ pour un atterrissage réussi (ou une pénalité de -100 pour un crash). L'épisode se termine soit à l'atterrissage, soit au crash, soit lors d'une sortie de la zone de simulation. Un épisode est considéré **résolu** lorsque la récompense atteint ≥ 200 .

Ce choix se justifie par plusieurs raisons : LunarLander est suffisamment complexe pour nécessiter un réglage soigneux des hyperparamètres, disponible en variantes discrète et continue (permettant de comparer des algorithmes de natures différentes), et constitue un benchmark de référence largement utilisé dans la littérature, facilitant la mise en perspective des résultats.

Trois algorithmes ont été sélectionnés pour couvrir les principales familles du RL profond :

- **DQN** (*Deep Q-Network*) (Mnih et al., 2015) : algorithme *off-policy* basé sur la Q-valeur, applicable uniquement aux espaces d'action discrets. Évalué sur la variante *LunarLander*.
- **PPO** (*Proximal Policy Optimization*) (Schulman ; Wolski et al., 2017) : algorithme *on-policy* de type acteur-critique, applicable aux deux types d'espaces d'action. Évalué sur les deux variantes.
- **SAC** (*Soft Actor-Critic*) (Haarnoja ; Zhou ; Abbeel et al., 2018) : algorithme *off-policy* à entropie maximale, conçu pour les espaces d'action continus. Évalué sur la variante *LunarLanderContinuous*.

Ce choix permet de comparer un algorithme à valeur (DQN), un algorithme *on-policy* (PPO) et un algorithme *off-policy* à entropie maximale (SAC), qui représentent trois approches fondamentalement différentes du problème d'apprentissage par renforcement.

8.2 Optimisation des hyperparamètres

8.2.1 Cadre général

L'optimisation des hyperparamètres est réalisée avec le framework **Optuna** (Akiba et al., 2019), en utilisant le sampler *Tree-structured Parzen Estimator* (TPE). TPE est un algorithme bayésien qui modélise séparément la distribution des bons et des mauvais hyperparamètres, et dirige la recherche vers les régions prometteuses de l'espace de configuration (Bergstra et al., 2011). Chaque configuration est évaluée sur **250 000 pas de temps** d'entraînement, suivie d'une évaluation déterministe sur **20 épisodes**. La métrique optimisée est la récompense moyenne obtenue lors de cette évaluation finale.

Ce budget de 250 000 steps, inférieur aux 500 000 steps retenus pour l'évaluation finale (Section 8.4.1), résulte d'un compromis pragmatique : avec une cible de 150 essais par combinaison algorithme/environnement, un budget de tuning plus élevé aurait rendu la campagne d'optimisation computationnellement prohibitive. Ce choix n'est cependant pas neutre : il repose sur l'hypothèse implicite que les hyperparamètres optimaux à 250 000 steps restent valides à 500 000 steps.

8.2.2 Élagage des essais non concluants

Afin de réduire le coût computationnel, un pruner médian est appliqué : il interrompt prématurément tout essai dont la performance intermédiaire (évaluée toutes les 10 000 steps) est inférieure à la médiane des essais précédents au même stade d'entraînement. Deux paramètres encadrent ce mécanisme :

- Les quinze premiers essais sont systématiquement menés à terme, sans élagage possible, afin de fournir au pruner une base de référence fiable avant d'activer le mécanisme.
- Au sein d'un essai, l'élagage ne peut s'activer qu'après 30 000 steps, laissant le temps à l'agent d'amorcer sa convergence avant d'être jugé.

Un essai interrompu par le pruner est enregistré avec l'état **PRUNED** dans la base Optuna et comptabilisé dans le total des essais effectués, au même titre qu'un essai complet. Seuls les essais ayant échoué pour des raisons techniques (crash, erreur mémoire) ne sont pas comptabilisés. La campagne cible au minimum **150 essais** par combinaison algorithme/environnement.

8.2.3 Hyperparamètres testés

Les tableaux suivants décrivent les hyperparamètres testés pour chaque algorithme, leur rôle et les plages de recherche utilisées. Pour l'ensemble des expériences, la politique neuronale repose sur des perceptrons multicouches (*MlpPolicy* dans Stable-Baselines3). L'espace d'observation de LunarLander étant un vecteur de huit dimensions, l'usage de réseaux convolutifs n'est pas pertinent dans ce contexte.

TABLE 2 – Hyperparamètres testés pour DQN

Hyperparamètre	Rôle	Plage / valeurs
<i>learning_rate</i>	Pas de gradient de l'optimiseur Adam. Une valeur trop élevée déstabilise l'entraînement ; trop faible, la convergence est lente.	$[10^{-5}, 10^{-3}]$
<i>buffer_size</i>	Taille du <i>replay buffer</i> . Un buffer plus grand améliore la diversité des transitions échantillonnées, au coût d'une empreinte mémoire plus importante.	$\{10^4, 5 \cdot 10^4, 10^5\}$
<i>learning_starts</i>	Nombre de pas avant le début des mises à jour. Permet de pré-remplir le buffer avec des transitions aléatoires pour stabiliser les premières itérations.	$\{0, 1000, 5000\}$
<i>batch_size</i>	Nombre de transitions échantillonnées à chaque mise à jour.	$\{32, 64, 128, 256\}$
<i>gamma</i>	Facteur d'actualisation des récompenses futures. Proche de 1 : l'agent est prévoyant ; proche de 0 : il privilégie les récompenses immédiates.	$\{0.90, \dots, 0.999\}$
<i>train_freq</i>	Fréquence des mises à jour en nombre de pas de temps.	$\{1, 4, 8, 16\}$
<i>target_update_interval</i>	Nombre de mises à jour entre deux synchronisations du réseau cible. Le <i>target network</i> stabilise l'apprentissage en fixant temporairement les valeurs cibles.	$\{100, 250, 500, 1000\}$
<i>exploration_fraction</i>	Fraction du training durant laquelle ε décroît de sa valeur initiale à sa valeur finale.	$[0.05, 0.5]$
<i>exploration_final_eps</i>	Valeur minimale de ε après la phase d'exploration. Correspond au taux d'actions aléatoires résiduel en exploitation.	$[0.01, 0.1]$

Suite page suivante

Suite du tableau 2

Hyperparamètre	Rôle	Plage / valeurs
<i>net_arch</i>	Architecture du réseau MLP — nombre et taille des couches cachées.	<i>small</i> [64,64], <i>medium</i> [256,256], <i>large</i> [400,300]

TABLE 3 – Hyperparamètres testés pour PPO

Hyperparamètre	Rôle	Plage / valeurs
<i>learning_rate</i>	Pas de gradient de l'optimiseur Adam.	$[10^{-5}, 10^{-3}]$
<i>n_steps</i>	Nombre de pas collectés avant chaque mise à jour de la politique, <i>rollout length</i> . Doit être supérieur à batch_size — les configurations invalides sont rejetées automatiquement lors du sampling.	{256, 512, 1024, 2048}
<i>batch_size</i>	Taille des mini-batches utilisés pour les mises à jour.	{32, 64, 128, 256, 512}
<i>n_epochs</i>	Nombre de passes sur les données collectées à chaque mise à jour Augmenter ce nombre améliore l'utilisation des données mais peut déstabiliser la politique.	{1, 3, 5, 10, 20}
<i>gamma</i>	Facteur d'actualisation des récompenses futures.	$\{0.90, \dots, 0.999\}$
<i>gae_lambda</i>	Paramètre λ du <i>Generalized Advantage Estimation</i> (GAE). Contrôle le compromis biais-variance dans l'estimation de l'avantage.	$\{0.80, \dots, 1.0\}$
<i>ent_coef</i>	Coefficient de la prime d'entropie. Encourage l'exploration en pénalisant les politiques trop déterministes.	$[10^{-8}, 0.1]$

Suite page suivante

Suite du tableau 3

Hyperparamètre	Rôle	Plage / valeurs
<i>clip_range</i>	Paramètre ε de la contrainte PPO. Limite l'écart entre l'ancienne et la nouvelle politique lors des mises à jour.	{0.1, 0.2, 0.3, 0.4}
<i>max_grad_norm</i>	Norme maximale du gradient — <i>gradient clipping</i> . Préviend les mises à jour explosives et stabilise l'entraînement.	[0.3, 5.0]
<i>vf_coef</i>	Coefficient de la perte du critique. Contrôle l'importance relative de l'erreur de prédiction de valeur par rapport à la perte de politique.	[0.1, 1.0]
<i>net_arch</i>	Architecture du réseau MLP acteur-critique.	<i>small</i> [64,64], <i>medium</i> [256,256], <i>large</i> [400,300]

TABLE 4 – Hyperparamètres testés pour SAC

Hyperparamètre	Rôle	Plage / valeurs
<i>learning_rate</i>	Pas de gradient commun aux réseaux acteur et critique.	$[10^{-5}, 10^{-3}]$
<i>buffer_size</i>	Taille du <i>replay buffer</i> .	$\{10^4, 5 \cdot 10^4, 10^5\}$
<i>learning_starts</i>	Nombre de pas avant le début des mises à jour.	{1000, 5000, 10000}
<i>batch_size</i>	Taille des mini-batches pour les mises à jour.	{32, 64, 128, 256, 512}
<i>gamma</i>	Facteur d'actualisation des récompenses futures.	$\{0.90, \dots, 0.999\}$
<i>tau</i>	Taux de mise à jour <i>soft</i> du réseau cible. À chaque pas : $\theta_{\text{cible}} \leftarrow \tau \theta + (1 - \tau) \theta_{\text{cible}}$.	{0.001, 0.005, 0.01, 0.02, 0.05}

Suite page suivante

Suite du tableau 4

Hyperparamètre	Rôle	Plage / valeurs
<i>ent_coef</i>	Coefficient d'entropie géré automatiquement (<i>auto</i>) : SAC ajuste ce coefficient à chaque pas de gradient pour maintenir une entropie cible $\mathcal{H}^* = -\dim(\mathcal{A})$, définie par Haarnoja ; Zhou ; Hartikainen et al., 2019 comme l'opposé de la dimension de l'espace d'action.	<i>auto</i>
<i>use_sde</i>	Active la <i>State-Dependent Exploration</i> (SDE). Le bruit d'exploration dépend de l'état courant ce qui peut améliorer l'exploration dans les espaces d'action continus. Ce bruit est maintenu pendant plusieurs steps consécutifs. Des trajectoires entières peuvent ainsi être explorées.	{True, False}
<i>target_update_interval</i>	Nombre de gradient steps entre deux mises à jour du réseau cible.	{1, 2, 4}
<i>net_arch</i>	Architecture des réseaux MLP acteur et critique.	<i>small</i> [64,64], <i>medium</i> [256,256], <i>large</i> [400,300]

8.3 Résultats de l'optimisation

Cette section présente les résultats obtenus pour chaque algorithme sur les deux variantes de l'environnement LunarLander, en mettant en lumière les configurations optimales identifiées et leur robustesse à travers les différentes seeds.

Le tableau 5 présente les scores obtenus par la meilleure configuration de chaque combinaison algorithme/environnement à l'issue de la phase de tuning. Ces scores correspondent à la récompense moyenne sur 20 épisodes déterministes, obtenue par le meilleur essai parmi les 150 évalués sur 250 000 steps. Ils constituent un indicateur du potentiel maximal de chaque configu-

ration, indépendamment de sa robustesse inter-seeds — qui fait l’objet de la section 8.4.2.

TABLE 5 – Score Optuna des meilleures configurations (récompense moyenne sur 20 épisodes déterministes, meilleur essai parmi 150)

Algorithme	Variante	Score Optuna
DQN	Discret	275.14
PPO	Discret	278.76
PPO	Continu	280.42
SAC	Continu	284.05

Les quatre configurations dépassent largement le seuil de résolution officiel de 200, ce qui confirme l’efficacité du protocole d’optimisation. SAC obtient le meilleur score sur la variante continue, tandis que les deux variantes PPO se situent dans un écart inférieur à 2 points, suggérant une robustesse de l’algorithme aux deux types d’espaces d’action sur cet environnement.

8.3.1 DQN — LunarLander discret

TABLE 6 – Hyperparamètres optimaux — DQN / discret (score Optuna : 275.14)

Hyperparamètre	Valeur optimale
<i>learning_rate</i>	4.73×10^{-4}
<i>buffer_size</i>	10^4
<i>learning_starts</i>	5 000
<i>batch_size</i>	64
<i>gamma</i>	0.99
<i>train_freq</i>	1
<i>target_update_interval</i>	250
<i>exploration_fraction</i>	0.253
<i>exploration_final_eps</i>	0.075

Suite du tableau 6

Hyperparamètre	Valeur optimale
<i>net_arch</i>	<i>medium</i> [256, 256]

La configuration optimale retenue pour DQN présente deux caractéristiques notables. D’une part, la taille du *replay buffer* (10^4) est inférieure aux valeurs typiques de la littérature (10^5 à 10^6 (Mnih et al., 2015)). Cette valeur s’explique vraisemblablement par le budget limité de la phase de tuning (250 000 steps) : avec un petit buffer, les transitions récentes dominent l’échantillonnage, ce qui peut accélérer la convergence à court terme mais reste à confirmer sur les 500 000 steps de l’évaluation finale. D’autre part, une fréquence de mise à jour à chaque pas de temps implique une mise à jour du réseau à chaque pas de temps, ce qui, combiné au petit buffer, produit un ratio mise-à-jour/données élevé. Cette configuration agressive a convergé efficacement sur cet environnement, mais introduit potentiellement une variance plus importante dans les gradients.

8.3.2 PPO — LunarLander discret

TABLE 7 – Hyperparamètres optimaux — PPO / discret (score Optuna : 278.76)

Hyperparamètre	Valeur optimale
<i>learning_rate</i>	2.37×10^{-4}
<i>n_steps</i>	2 048
<i>batch_size</i>	256
<i>n_epochs</i>	20
<i>gamma</i>	0.995
<i>gae_lambda</i>	0.98
<i>ent_coef</i>	1.55×10^{-2}
<i>clip_range</i>	0.4
<i>max_grad_norm</i>	2.13

Suite du tableau 7

Hyperparamètre	Valeur optimale
<i>vf_coef</i>	0.889
<i>net_arch</i>	<i>large</i> [400, 300]

La configuration PPO discrète se distingue par un seuil d'écèlement ϵ de 0.4, valeur maximale de la plage explorée et nettement supérieure à la valeur standard de 0.2 (Schulman ; Wolski et al., 2017). Un seuil d'écèlement élevé autorise des mises à jour de politique plus importantes à chaque gradient step, tout en conservant la garantie de stabilité propre à PPO grâce au mécanisme de clipping. Cette valeur, associée à vingt époques d'optimisations sur des rollouts de 2048 transitions, indique que PPO effectue $\lfloor 2048/256 \rfloor = 8$ mini-batches par époque, soit 160 mises à jour du réseau pour chaque rollout de 2048 transitions.

8.3.3 PPO — LunarLanderContinuous

TABLE 8 – Hyperparamètres optimaux — PPO / continu (score Optuna : 280.42)

Hyperparamètre	Valeur optimale
<i>learning_rate</i>	6.42×10^{-5}
<i>n_steps</i>	2048
<i>batch_size</i>	32
<i>n_epochs</i>	10
<i>gamma</i>	0.995
<i>gae_lambda</i>	0.98
<i>ent_coef</i>	1.27×10^{-4}
<i>clip_range</i>	0.4
<i>max_grad_norm</i>	1.65
<i>vf_coef</i>	0.105

Suite du tableau 8

Hyperparamètre	Valeur optimale
<i>net_arch</i>	<i>large</i> [400, 300]

La comparaison entre les deux variantes PPO révèle une convergence vers des valeurs communes pour plusieurs hyperparamètres structurels (la longueur de rollout (2048 steps), le paramètre γ du GAE (0.98), le seuil d'écrtage ϵ (0.4) et l'architecture du réseau (large)), ce qui suggère que ces choix sont robustes à la nature de l'espace d'action sur cet environnement. En revanche, deux hyperparamètres diffèrent significativement entre les deux variantes : le taux d'apprentissage est quatre fois plus faible pour la variante continue (6.42×10^{-5} contre 2.37×10^{-4}), et le coefficient d'entropie est deux ordres de grandeur inférieur (1.27×10^{-4} contre 1.55×10^{-2}). Ces différences indiquent que l'optimisation d'une politique gaussienne en espace continu requiert des mises à jour plus prudentes et une pression d'exploration entropique moindre que dans le cas discret.

8.3.4 SAC — LunarLanderContinuous

TABLE 9 – Hyperparamètres optimaux — SAC / continu (score Optuna : 284.05)

Hyperparamètre	Valeur optimale
<i>learning_rate</i>	9.95×10^{-4}
<i>buffer_size</i>	10^5
<i>learning_starts</i>	1 000
<i>batch_size</i>	512
<i>gamma</i>	0.99
<i>tau</i>	0.05
<i>ent_coef</i>	<i>auto</i>
<i>use_sde</i>	<i>True</i>
<i>target_update_interval</i>	2

Suite du tableau 9

Hyperparamètre	Valeur optimale
<i>net_arch</i>	<i>medium</i> [256, 256]

SAC obtient le meilleur score Optuna (284.05) avec une architecture de réseau *medium* [256, 256], plus petite que celle retenue par PPO (*large* [400, 300]), ce qui illustre l’efficacité par paramètre propre aux algorithmes *off-policy* bénéficiant du *replay buffer*. Deux hyperparamètres méritent d’être soulignés. D’une part, le taux d’interpolation τ atteint sa valeur maximale testée (0.05), dix fois supérieure à la valeur standard de 0.005 (Haarnoja ; Zhou ; Abbeel et al., 2018) : cette mise à jour rapide du réseau cible favorise une convergence accélérée sur un environnement de faible dimensionnalité comme LunarLander, où le risque de divergence lié à un réseau cible instable est limité. D’autre part, `use_sde = True` confirme l’intérêt de la *State-Dependent Exploration* pour les espaces d’action continus : le bruit d’exploration structuré en fonction de l’état permet une exploration plus cohérente que le bruit gaussien indépendant de l’état.

8.4 Évaluation finale multi-seeds

8.4.1 Protocole d’évaluation finale

Une fois les hyperparamètres optimaux identifiés pour chaque combinaison algorithme/environnement, une évaluation finale est conduite pour mesurer la robustesse des configurations retenues. Chaque modèle est réentraîné **from scratch** sur **5 seeds indépendantes** ($\{42, 123, 456, 789, 1337\}$) avec 500 000 pas de temps d’entraînement, puis évalué sur 20 épisodes.

Évaluation déterministe. L’évaluation est conduite en mode **déterministe**, ce qui diffère du comportement adopté durant l’entraînement selon l’algorithme :

- **PPO** : pendant l’entraînement, PPO échantillonne ses actions depuis

une distribution de probabilités (gaussienne pour le continu, catégorielle pour le discret) afin d’explorer. En mode déterministe, l’action retenue est celle de probabilité maximale, sans aucun échantillonnage.

- **SAC** : de même, SAC paramétrise une distribution gaussienne et échantillonne lors de l’entraînement pour maximiser l’entropie. En mode déterministe, il retourne la moyenne de cette distribution, ce qui correspond à la politique la plus probable sans bruit d’exploration.
- **DQN** : le mode déterministe consiste à sélectionner systématiquement l’action de Q-valeur maximale (argmax), avec $\varepsilon = 0$. C’est déjà le comportement naturel de DQN lors de l’exploitation ; le mode déterministe ne change donc pas fondamentalement la politique évaluée.

Ce choix d’évaluation déterministe est standard dans la littérature RL : il mesure la politique *apprise* sans bruit d’exploration, offrant une estimation plus fidèle des performances en déploiement réel.

Métriques rapportées. Les métriques suivantes sont calculées sur les 20 épisodes d’évaluation, puis agrégées sur les 5 seeds :

- Récompense moyenne et écart-type inter-seeds,
- Récompense minimale et maximale observées,
- Taux de succès : fraction d’épisodes avec une récompense ≥ 200 (seuil de résolution officiel de LunarLander),
- Nombre de seeds pour lesquelles la récompense moyenne atteint le seuil de résolution.

Cette procédure multi-seeds permet de distinguer les configurations robustes — qui convergent systématiquement — des configurations qui réussissent de manière occasionnelle en raison de la stochasticité de l’entraînement.

8.4.2 Résultats de l’évaluation multi-seeds

Les résultats de l’évaluation multi-seeds sont présentés dans le tableau 10. Chaque configuration optimale est réentraînée sur 5 seeds indépendantes ($\{42, 123, 456, 789, 1337\}$) pendant 500 000 steps, puis évaluée de manière déterministe sur 20 épisodes. Les métriques rapportées sont la récompense

moyenne et son écart-type inter-seeds (écart-type échantillonnal, $n - 1$), les récompenses extrêmes observées, le taux de succès (fraction d'épisodes avec une récompense ≥ 200) et le nombre de seeds pour lesquelles la récompense moyenne dépasse le seuil de résolution.

TABLE 10 – Résultats de l'évaluation multi-seeds (500 000 steps, récompense moyenne sur 20 épisodes déterministes, écart-type échantillonnal $n - 1$)

Algo.	Env.	Moyenne $\pm$ Écart-type	Min	Max	Succès	Seeds
DQN	Discret	145.19 ± 45.76	101.26	231.89	39%	1/5
PPO	Discret	218.58 ± 50.72	143.95	278.76	77%	3/5
PPO	Continu	208.67 ± 50.17	159.60	278.96	74%	2/5
SAC	Continu	233.44 ± 52.50	132.67	281.01	79%	4/5

SAC et PPO — configurations robustes. SAC obtient la meilleure récompense moyenne (233.44 ± 52.50) et le taux de succès le plus élevé (79%, 4 seeds sur 5 résolues), confirmant la supériorité de l'algorithme à entropie maximale sur la variante continue. PPO affiche des performances solides dans les deux variantes, avec des récompenses moyennes supérieures au seuil de résolution (218.58 en discret, 208.67 en continu) et des taux de succès proches de 75%. La variante discrète résout légèrement plus de seeds (3/5 contre 2/5), bien que les scores moyens soient proches, ce qui suggère une variance inter-seeds plus favorable en espace discret.

DQN — fragilité de la configuration optimale. DQN présente un écart marqué entre son score Optuna (275.14) et sa récompense moyenne en évaluation finale (145.19), soit une chute de plus de 130 points. Cet écart s'explique vraisemblablement par la nature de la configuration retenue : un *replay buffer* de 10^4 transitions combiné à une fréquence de mise à jour maximale constituent une configuration agressive, optimisée pour converger rapidement sur 250 000 steps de tuning, mais qui ne se généralise pas sur les 500 000 steps de l'évaluation finale. Le ratio élevé mise-à-jour/données induit une variance importante dans les gradients, ce qui se traduit par une instabilité inter-seeds (écart-type de 45.76, une seule seed résolue sur 5).

Écarts-types élevés — limite de l'évaluation à 5 seeds. Les écarts-types inter-seeds sont importants pour tous les algorithmes (45–52 points), ce qui reflète la stochasticité inhérente à l'entraînement par renforcement. Avec seulement 5 seeds, ces estimations restent bruitées. Cette phase ayant pour vocation la validation du protocole d'optimisation plutôt qu'une évaluation exhaustive, ce nombre de seeds est néanmoins suffisant pour dégager les tendances principales.

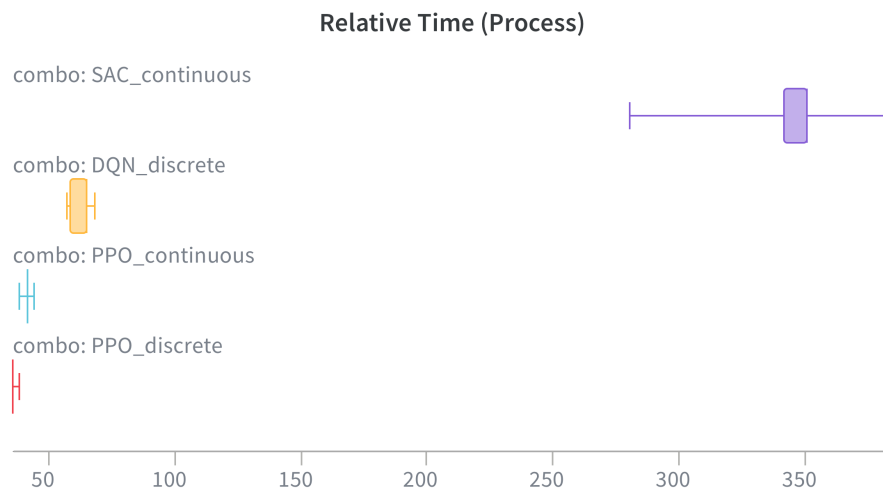


FIGURE 5 – Distribution du temps de calcul (en minutes) pour chaque combinaison algorithme/environnement, sur les 5 seeds indépendantes.

La figure 5 illustre la distribution des temps de calcul inter-seeds. Cette métrique correspond au temps de processus observé pendant l'entraînement et constitue une approximation du coût temporel de convergence, sans coïncider exactement avec le temps de convergence algorithmique.

SAC requiert globalement un temps d'entraînement plus important que PPO et DQN, ce qui s'explique par le coût computationnel de ses mises à jour : SAC maintient simultanément deux réseaux critic et un acteur, et calcule à chaque step la mise à jour du coefficient d'entropie adaptatif — un coût par update structurellement plus élevé que celui des autres algorithmes, indépendamment de la fréquence des mises à jour. Il est important de noter que ces tests ont été réalisés sur un cluster aux ressources partagées ; les expériences n'ont donc pas nécessairement bénéficié de conditions matérielles

strictement identiques, ce qui peut introduire une variance supplémentaire dans les temps de calcul observés. Cependant, la tendance générale reste claire : SAC est plus coûteux en temps d’entraînement que PPO et DQN, ce qui doit être pris en compte dans le choix de l’algorithme en fonction des contraintes de ressources disponibles.

Synthèse. Ces résultats mettent en évidence deux enseignements principaux. D’une part, le score Optuna n’est pas un prédicteur fiable de la robustesse inter-seeds : DQN en est l’illustration la plus nette, avec le deuxième meilleur score de tuning mais les performances finales les plus faibles. D’autre part, SAC et PPO représentent des compromis distincts : SAC offre les meilleures performances sur la variante continue au prix d’un coût computationnel plus élevé, tandis que PPO, applicable aux deux types d’espaces d’action avec des performances stables, constitue un choix plus polyvalent pour des contextes aux ressources limitées.

9 Phase II - Apprentissage sur Rocket League

Pour cette phase 2, le but est de travailler sur un environnement bien plus complexe que LunarLander, nécessitant la maîtrise de mécanique avancées et une stratégie à long terme. Rocket League a été retenu comme environnement de complexité avancée. Il s’agit d’un jeu de football avec des voitures développé par Psyonix, qui constitue un défi particulièrement riche pour l’apprentissage par renforcement en raison de sa physique, de la grande variété de situations de jeu possibles et de la nécessité de coordonner des séquences d’actions complexes pour être compétitif.

9.1 Environnement de jeu

Rocket League est un jeu de football avec des voitures dans lequel les joueurs contrôlent des véhicules pour frapper une balle et marquer des buts dans le camp adverse. L’environnement est particulièrement exigeant pour un agent de RL pour plusieurs raisons. D’une part, le jeu tourne nativement à 120 images par seconde (RLGym, 2020a), ce qui implique qu’un agent devrait

théoriquement prendre 120 décisions par seconde pour interagir avec l'environnement à sa pleine résolution temporelle. D'autre part, les mécaniques de jeu sont nombreuses et interdépendantes : les voitures peuvent récupérer du boost sur le terrain, effectuer des sauts simples ou doubles, réaliser des *flips* (rotations rapides en l'air pouvant conférer une impulsion de vitesse supplémentaire), et coller aux murs ou au plafond de l'arène.

Le jeu peut se pratiquer en mode 1v1, 2v2, 3v3 ou 4v4. Afin de limiter la complexité liée à la coordination entre plusieurs agents, ce travail se concentre exclusivement sur le mode 1v1, où un seul agent affronte un adversaire unique⁶. Ce cadre permet d'isoler les défis liés à l'apprentissage individuel — contrôle du véhicule, gestion du boost, positionnement tactique — sans introduire la dimension supplémentaire de la collaboration multi-agents.

Dans cette phase, ce n'est pas le jeu officiel qui est utilisé, mais un environnement de simulation appelé RocketSim (Section 7.5.1), qui reproduit fidèlement la physique du jeu officiel tout en permettant un entraînement en *headless mode* — sans rendu graphique — ce qui accélère considérablement les cycles d'expérimentation. RLGym fournit l'interface entre RocketSim et les algorithmes d'apprentissage par renforcement, en exposant les observations, les récompenses et les actions dans un format standardisé compatible avec les bibliothèques de RL modernes (RLGym, 2020b).

9.1.1 Espace d'actions

Le vecteur d'action complet dans RLGym est composé de huit dimensions, décrites dans le tableau 11. Cinq d'entre elles sont continues (throttle, steer, pitch, yaw, roll) et trois sont discrètes (jump, boost, handbrake).

L'espace d'actions brut, en considérant toutes les combinaisons discrétisées possibles de ces huit dimensions, atteint 1 944 combinaisons. Ce chiffre résulte du produit cartésien entre les trois actions discrètes, chacune admettant deux valeurs possibles ($2^3 = 8$), et les cinq dimensions continues, ici discrétisées en trois niveaux représentatifs ($-1, 0, 1$) afin de rendre l'espace d'actions

6. Dans notre contexte, il s'agit de lui-même

Action	Plage	Type	Description
Throttle	$[-1,1]$	Continu	Accélération/décélération du véhicule
Steer	$[-1,1]$	Continu	Direction de braquage au sol
Pitch	$[-1,1]$	Continu	Rotation verticale avant/arrière (en l'air)
Yaw	$[-1,1]$	Continu	Rotation horizontale (en l'air)
Roll	$[-1,1]$	Continu	Rotation latérale (en l'air)
Jump	$0,1$	Discret	Saut simple ou double
Boost	$0,1$	Discret	Utilisation du boost
Handbrake	$0,1$	Discret	Frein à main (dérapage)

TABLE 11 – Espace d'actions dans RLGym (RLGym, 2020b)

dénombrable ($3^5 = 243$) :

$$|\mathcal{A}| = \underbrace{2^3}_{\text{actions discrètes}} \times \underbrace{3^5}_{\text{actions continues discrétisées}} = 8 \times 243 = 1\,944 \quad (22)$$

Entraîner un agent sur un espace aussi vaste ralentirait considérablement la convergence, car une grande partie de ces combinaisons est redondante ou sans effet mécanique selon le contexte (par exemple, utiliser le boost sans accélérer simultanément avec le throttle, ou solliciter conjointement le steer et le yaw, deux actions dont les domaines d'effet — sol et air respectivement — ne se recouvrent pas).

Pour remédier à cela, une table de correspondance (*lookup table*) est utilisée via l'outil de discrétisation de RLGym (RLGym, 2020b), qui réduit l'espace d'actions à **90 combinaisons discrètes** en ne conservant que les actions mécaniquement pertinentes. Ces 90 actions se décomposent en deux blocs :

Actions au sol (18 actions). Elles couvrent toutes les combinaisons utiles de throttle $\in \{-1, 0, 1\}$, steer $\in \{-1, 0, 1\}$, boost $\in \{0, 1\}$ et handbrake $\in \{0, 1\}$, soit $3 \times 3 \times 2 \times 2 = 36$ combinaisons brutes. Sous la contrainte que le boost n'est activé que si le throttle est au maximum (boost = 1 $\Rightarrow$

throttle = 1), ce qui correspond au comportement physique réel du jeu, les combinaisons où boost = 1 et throttle $\neq$ 1 sont écartées, ramenant le nombre d'actions au sol à 18.

Actions aériennes (72 actions). Elles couvrent les combinaisons de pitch, yaw, roll, jump et boost lorsque le véhicule est en l'air. Les doublons avec les actions au sol sont supprimés, ainsi que les combinaisons yaw+jump (remplacées par roll pour les flips horizontaux), ce qui ramène ce bloc à 72 actions.

Au total : $|\mathcal{A}| = 18 + 72 = 90$ actions discrètes.

Ce choix d'un espace discret réduit présente deux avantages majeurs. D'une part, il est directement compatible avec PPO, dont la couche de sortie applique une distribution catégorielle sur les actions discrètes (Section 5.8.2). D'autre part, il réduit significativement la complexité de l'exploration, ce qui favorise une convergence plus rapide — particulièrement important dans un environnement qui nécessite plusieurs milliards de pas d'entraînement pour atteindre un niveau de jeu compétitif.

SAC, en revanche, repose nativement sur un espace d'actions continu (Section 5.8.3) : son application à cet environnement a donc nécessité l'utilisation de l'espace d'actions continu brut à huit dimensions, sans passage par la table de correspondance discrète décrite ci-dessus. Cette différence fondamentale d'espace d'actions entre les deux algorithmes, et son impact sur la comparaison, est discutée en Section 9.6.

Répétition d'actions (*action repeat*). À la fréquence native de 120 ticks par seconde, un agent devrait théoriquement prendre 120 décisions distinctes par seconde. En pratique, cette granularité temporelle est contre-productive pour l'apprentissage : elle génère un signal de récompense extrêmement bruité, rend le problème d'assignation de crédit (*credit assignment*) particulièrement difficile — il devient ardu pour l'agent de déterminer quelle action parmi les 120 a réellement contribué à une récompense donnée — et s'éloigne considérablement de la fréquence décisionnelle humaine.

Pour remédier à cela, un mécanisme de répétition d'actions est appliqué avec un facteur de 8 ticks, ramenant la fréquence décisionnelle effective de

l’agent à $120/8=15$ Hz. Chaque action sélectionnée par la politique est ainsi maintenue pendant 8 ticks consécutifs avant qu’une nouvelle décision soit prise. Ce principe, connu sous le nom de *frame skipping* dans la littérature du Deep RL appliqué aux jeux vidéo, a été introduit et validé empiriquement par Mnih et al. (2015) dans le cadre d’Atari, où un facteur de répétition de 4 frames est appliqué. Il est depuis devenu une pratique standard dans les environnements à haute fréquence de simulation.

9.1.2 Espace d’observation

L’espace d’observation fournit à l’agent un vecteur d’état normalisé décrivant la situation de jeu à chaque pas de temps. Toutes les valeurs sont normalisées pour être contenues dans l’intervalle $[-1, 1]$, ce qui est une pratique standard en Deep RL pour stabiliser l’apprentissage et éviter que certaines dimensions ne dominent numériquement les autres (Engstrom et al., 2020; Stable-Baselines3, 2021).

Le vecteur d’observation décrit chaque entité de jeu — la voiture de l’agent, la voiture adverse et la balle — à travers les informations suivantes :

- **Position** (x, y, z) normalisée par les dimensions de l’arène.
- **Orientation** du véhicule dans l’espace 3D, encodée sous forme des vecteurs *forward*, *up* et *right*.
- **Vitesse linéaire** (v_x, v_y, v_z) normalisée par la vitesse maximale atteignable.
- **Vitesse angulaire** $(\omega_x, \omega_y, \omega_z)$ normalisée par la vitesse angulaire maximale.
- **Quantité de boost** disponible, normalisée sur $[0, 1]$.
- **Variables d’état booléennes** : contact avec le sol, disponibilité du saut, état de démolition du véhicule.

Cet espace d’observation est dit *omniscient* : l’agent dispose en permanence d’informations complètes sur l’ensemble de l’environnement, ce qui le place dans des conditions bien plus favorables qu’un joueur humain. En particulier, la position et la vitesse exactes de la balle, ainsi que le positionnement et le niveau de boost de l’adversaire, sont accessibles à tout moment — des informations qu’un joueur humain ne peut qu’estimer visuellement et impar-

faitement.

Par ailleurs, l'état du jeu est réinitialisé après 5 minutes (durée d'une partie normale de Rocket League), lorsqu'un but est marqué ou lorsqu'aucun des deux joueurs n'a touché la balle pendant 30 secondes. Ces conditions de terminaison sont cohérentes avec les règles officielles du jeu et permettent de structurer les épisodes d'entraînement de manière réaliste.

Génération d'états initiaux aléatoires et exploration. Par défaut, une partie de Rocket League débute systématiquement par une phase de coup d'envoi (*kickoff*), caractérisée par un positionnement parfaitement symétrique des joueurs et une balle immobile au centre du terrain. Si ce cadre est conforme aux règles officielles, il s'avère restrictif pour l'exploration initiale de l'agent. Ce dernier se retrouve contraint de sur-apprendre la gestion de cette situation ultra-spécifique avant de pouvoir explorer des phases de jeu plus dynamiques, ce qui ralentit considérablement la convergence globale.

Pour pallier cette limite, un mécanisme de modification d'état (*state mutator*) a été introduit lors des phases de réinitialisation. À l'exception des épisodes explicitement dédiés aux phases de coup d'envoi, ce module applique des perturbations stochastiques aux configurations initiales : la balle et les véhicules sont positionnés de manière aléatoire dans l'arène avec des vitesses initiales variables. Cette approche présente un double avantage théorique et pratique. D'une part, elle force l'agent à généraliser ses compétences à une grande diversité de situations géométriques et cinétiques dès le début de l'entraînement, évitant ainsi une spécialisation précoce. D'autre part, elle favorise une exploration approfondie de l'espace d'états — notamment les situations défensives proches des buts ou les trajectoires aériennes — qui auraient été extrêmement difficiles à découvrir pour une politique purement aléatoire partant d'un *kickoff* standard.

9.2 Objectif du jeu et problème du reward sparse

L'objectif fondamental de Rocket League est de marquer davantage de buts que l'adversaire avant la fin du temps réglementaire, en poussant la balle dans le but adverse à l'aide du véhicule. Cet objectif unique et bien défini

suggère, à première vue, une formulation triviale de la fonction de récompense : attribuer une récompense positive lorsqu'un but est marqué, et une récompense négative symétrique lorsqu'un but est concédé.

Cette formulation minimaliste se heurte cependant au problème classique du *reward sparse* (récompense rare) en apprentissage par renforcement. Ce signal isolé constitue alors le seul indicateur d'apprentissage disponible, et celui-ci ne survient qu'occasionnellement au cours d'un épisode — voire jamais durant les premières phases de l'entraînement, lorsque la politique de l'agent est encore proche d'un comportement aléatoire. Dans un espace d'états et d'actions de grande dimension, la probabilité qu'une séquence d'actions aléatoires mène fortuitement à un but est extrêmement faible. L'agent ne reçoit alors quasiment aucun signal exploitable pour orienter l'amélioration de sa politique, ce qui se traduit en pratique par une absence totale d'apprentissage observable, même après plusieurs millions de pas d'entraînement.

Ce problème n'est pas spécifique à Rocket League : il est documenté de manière générale dans la littérature du RL appliqué à des environnements dont l'objectif final est temporellement éloigné des actions élémentaires de l'agent (Narvekar et al., 2020). Deux familles de solutions complémentaires sont généralement proposées pour pallier cette rareté du signal. Le *reward shaping* agit sur la **composition** de la fonction de récompense elle-même : à un instant donné de l'entraînement, le signal sparse de l'objectif final est enrichi de signaux intermédiaires plus denses, sans changer la nature de la tâche à résoudre. Le *curriculum learning*, en revanche, agit sur la **progression temporelle** de l'entraînement : il consiste à faire évoluer la tâche elle-même — et donc potentiellement la fonction de récompense associée — au fil du temps, en structurant l'apprentissage en une séquence de sous-tâches de complexité croissante.

Ces deux approches sont complémentaires plutôt que concurrentes, et c'est leur combinaison qui structure la démarche méthodologique adoptée dans ce travail, détaillée dans les sections suivantes.

9.3 Reward shaping

Le *reward shaping* désigne la pratique consistant à augmenter une fonction de récompense sparse avec des signaux intermédiaires plus denses, dans le but d'accélérer et de faciliter l'apprentissage. Plutôt que de se limiter au signal binaire du but, des récompenses continues sont introduites pour guider progressivement l'agent vers des sous-comportements jugés nécessaires à l'objectif final : se rapprocher de la balle, la toucher, l'orienter vers le but adverse, etc.

Le fondement théorique de cette approche repose sur les travaux de Ng et al. (1999), qui démontrent qu'un *reward shaping* construit à partir d'une fonction de potentiel (*potential-based reward shaping*) préserve la politique optimale du problème original : l'ajout de récompenses intermédiaires bien formulées ne modifie pas la solution recherchée par l'agent, mais accélère uniquement la vitesse à laquelle celle-ci est découverte. Si les récompenses introduites dans ce travail n'ont pas systématiquement été formalisées sous une forme strictement potential-based, ce résultat théorique a néanmoins guidé la conception générale : chaque récompense intermédiaire est pensée comme un indice complémentaire à l'objectif final, et non comme un substitut à celui-ci.

En pratique, la conception de ces signaux intermédiaires s'est avérée être un processus itératif et empirique, ajusté au fil de nombreuses itérations d'entraînement. Deux catégories fondamentales de récompenses peuvent être distinguées :

- **Récompenses basées sur les événements** (*event-based*), telles qu'un signal de but ou de démolition de l'adversaire, qui attribuent un signal ponctuel et discret lors d'un événement précis du jeu. Ce type de récompense est par nature sparse : le signal n'est émis qu'au moment de l'événement, et reste nul le reste du temps.
- **Récompenses basées sur l'état** (*state-based*), telles que la vitesse du joueur vers la balle, l'alignement du regard vers la balle ou la gestion de l'énergie cinétique, qui évaluent à chaque pas de temps une propriété continue de l'état du jeu ou du véhicule. Contrairement aux récompenses événementielles, ce type de signal est dense : il fournit un retour à chaque pas de temps, qu'un comportement particulier se

produise ou non.

Cette distinction structure l'essentiel de la démarche de reward shaping adoptée dans ce travail : le signal sparse d'un but marqué constitue l'objectif final recherché, tandis que l'ensemble des récompenses basées sur l'état viennent densifier le signal d'apprentissage pour guider l'agent vers ce but. Leur introduction progressive, dans un ordre cohérent avec la complexité croissante des comportements visés, est détaillée en Section 9.4.

9.4 Curriculum d'apprentissage

La conception de la fonction de récompense a suivi une progression structurée en plusieurs phases successives, chacune introduisant un niveau de complexité supplémentaire par rapport à la précédente. Cette démarche s'inscrit dans le paradigme du *curriculum learning* (Section 9.2) : plutôt que de confronter directement l'agent à l'objectif final (marquer des buts), chaque phase vise l'acquisition d'une compétence intermédiaire, en s'appuyant sur les comportements déjà acquis lors des phases précédentes. Le signal de récompense de chaque phase conserve les composantes des phases antérieures, auxquelles s'ajoutent de nouveaux signaux orientés vers la compétence visée – garantissant ainsi que les comportements déjà appris ne sont pas dégradés par l'introduction de nouveaux objectifs. L'application concrète de ce principe à l'environnement étudié, ainsi que le détail des fonctions de récompense associées à chaque phase, est présentée en Section 9.6.

Cette progression par phases s'est avérée nécessaire en pratique : les premiers essais d'entraînement, menés avec l'ensemble des signaux de récompense introduits simultanément dès le début, semblaient présenter une convergence plus lente vers un comportement de poursuite de la balle cohérent, par rapport à l'introduction progressive et séquentielle adoptée par la suite. L'agent semblait alors avoir davantage de difficulté à isoler le comportement pertinent au sein d'un signal global plus complexe.

Il est par ailleurs notable que certains comportements mécaniques avancés du jeu – tels que le *flick* (mouvement consistant à projeter brusquement la balle vers l'avant depuis une position de contrôle) – ont émergé spontanément chez l'agent sans qu'aucune récompense spécifique n'ait été conçue pour

les encourager explicitement (Paulino, 2026a). Ces comportements résultent vraisemblablement de l’optimisation conjointe des signaux plus généraux de la fonction de récompense (toucher la balle avec vitesse, l’orienter vers le but), suggérant qu’une fonction de récompense suffisamment bien conçue peut conduire à l’émergence de stratégies non anticipées par le concepteur.

9.5 Reward hacking et limites du reward shaping

Au-delà de la conception initiale des signaux de récompense, l’ensemble du processus de reward shaping mené dans ce travail a été marqué par l’apparition récurrente de comportements exploitant des failles ou ambiguïtés non anticipées dans la formulation des récompenses. Ce phénomène, désigné dans la littérature sous le terme de *reward hacking*, constitue une limite fondamentale du reward shaping : un signal de récompense mal spécifié, même bien intentionné, peut admettre une solution qui le maximise formellement tout en s’écartant de l’intention réelle du concepteur (Amodei et al., 2016). Cette limite prolonge les travaux de Ng et al. (1999) sur la préservation théorique de la politique optimale sous reward shaping (*policy invariance*) : même lorsque le shaping respecte ce cadre théorique idéal, la formulation concrète des récompenses reste, en pratique, sujette à des ambiguïtés que l’agent peut exploiter.

Plusieurs occurrences de ce phénomène ont été observées au cours de l’entraînement, résumées dans le tableau 12. Dans chaque cas, l’identification du comportement exploité a nécessité une observation visuelle directe du comportement de l’agent (via *RocketSim Vis*, Section 7.5.2) plutôt qu’une simple analyse des courbes de récompense agrégées, ces dernières ne révélant pas nécessairement la nature du comportement sous-jacent.

Ces observations soulignent une caractéristique pratique essentielle du reward shaping en environnement compétitif multi-agent : un signal de récompense individuel, aussi bien intentionné soit-il, peut être détourné par une stratégie collaborative implicite entre agents lorsqu’il n’est pas formulé de manière intrinsèquement compétitive. Cette problématique illustre une caractéristique fondamentale des environnements compétitifs multi-agents tels que le 1v1 retenu dans ce travail : l’interaction entre deux agents partageant le même état et observant chacun une récompense distincte correspond formellement

Concept de récompense	Comportement exploité	Correction conceptuelle	Vidéos
Récompense de contact simple (binaire)	Les deux agents se coordonnent pour toucher la balle mutuellement à très faible vitesse, maximisant le nombre de contacts plutôt que leur pertinence tactique.	Pondération du contact par l'impulsion et l'accélération imprimées à la balle. Retour à un point de sauvegarde antérieur.	(Paulino, 2026c)
Récompense de vitesse de la balle vers le but (asymétrique)	Les deux agents collaborent pour faire osciller la balle entre les deux buts, maximisant simultanément leur gain individuel sans chercher à marquer.	Reformulation de la récompense sous forme d'un jeu à somme nulle (<i>zero-sum reward</i>).	(Paulino, 2026b)

TABLE 12 – Exemples de *reward hacking* observés et corrections conceptuelles apportées

à un *jeu de Markov* (*Markov game*) à somme nulle (Littman, 1994). Dans ce cadre théorique, un agent cherche à maximiser son propre retour cumulé tandis que l'autre cherche, de manière équivalente, à le minimiser — ce qui garantit formellement qu'aucune stratégie collaborative implicite entre les deux agents ne puisse être mutuellement avantageuse à l'équilibre.

La formulation des récompenses compétitives sous forme *zero-sum* dans ce travail constitue ainsi une application directe de ce principe théorique au contexte du *self-play* : chaque agent étant entraîné contre une copie de sa propre politique, la symétrie de la récompense garantit que l'amélioration de l'un se traduit nécessairement par une pression équivalente sur l'autre, évitant la convergence vers des comportements collaboratifs dégénérés tels que ceux observés dans le tableau 12. Cette correction a conduit à l'adoption systématique de récompenses *zero-sum* pour l'ensemble des signaux de nature compétitive introduits dans les phases ultérieures du curriculum.

9.6 Méthodologie de comparaison entre PPO et SAC

Conformément au principe de curriculum décrit en Section 9.4, l’entraînement a été structuré en trois phases successives : (I) acquisition du contrôle de base du véhicule et du contact avec la balle, (II) apprentissage de l’objectif de score, et (III) perfectionnement des mécaniques avancées du jeu. Ce curriculum a été appliqué de manière identique à PPO et SAC, afin de permettre une comparaison empirique entre les deux algorithmes. Les fonctions de récompense associées à chaque phase, ainsi que leurs poids respectifs, sont détaillées dans le tableau 13.

Phase	Objectif	Composantes de récompense (poids)	Budget
I	Contrôle de base et contact avec la balle	<i>AdvancedTouchReward</i> (5), <i>VelocityPlayerToBallReward</i> (1), <i>InAirReward</i> (0.1), <i>FaceBallReward</i> (0.1)	~100M
II	Apprentissage du score	<i>GoalReward</i> (150), <i>AdvancedTouchReward</i> (5), <i>VelocityPlayerToBallReward</i> (1), <i>VelocityBallToGoalReward_ZS</i> (6), <i>BoostKeepReward</i> (3), <i>InAirReward</i> (0.1), <i>FaceBallReward</i> (0.1)	~1B
III	Perfectionnement mécanique (boost, aérien, kickoff)	<i>GoalReward</i> (300), <i>AdvancedTouchReward</i> (7.5), <i>VelocityPlayerToBallReward</i> (1), <i>VelocityBallToGoalReward_ZS</i> (10), <i>InAirReward</i> (0.1), <i>KickoffReward</i> (5), <i>BoostKeepReward_ZS</i> (3), <i>EnergyReward</i> (0.05), <i>AerialDistanceReward</i> (1.5)	–

TABLE 13 – Curriculum de récompenses utilisé pour la comparaison PPO/SAC

9.6.1 Phase I – Contrôle de base et contact avec la balle

La première phase vise l’acquisition des compétences fondamentales de contrôle du véhicule et de contact avec la balle, indépendamment de toute notion de but. L’objectif est que l’agent apprenne à se déplacer efficacement, à s’orienter vers la balle et à entrer en contact avec elle de manière répétée. La fonction de récompense associée combine un signal de contact (*AdvancedTouchReward*),

un signal de rapprochement vers la balle (*VelocityPlayerToBallReward*), ainsi que deux signaux secondaires encourageant respectivement le jeu aérien (*InAirReward*) et l'orientation vers la balle (*FaceBallReward*).

9.6.2 Phase II – Apprentissage du score

La deuxième phase introduit l'objectif final du jeu – marquer des buts – tout en conservant les signaux de la phase précédente afin de ne pas dégrader les comportements déjà appris (Section 9.4). Le signal événementiel *GoalReward* y est introduit avec un poids dominant, accompagné d'un signal dense orientant la trajectoire de la balle vers le but adverse (*VelocityBallToGoalReward_ZS*) et d'un premier signal de gestion du boost (*BoostKeepReward*).

Conformément à la discussion menée en Section 9.5, *VelocityBallToGoalReward* est formulée sous forme *zero-sum* (*VelocityBallToGoalReward_ZS*) plutôt que sous sa forme individuelle brute. Cette précaution découle directement du comportement de *reward hacking* observé en self-play avec une formulation asymétrique de ce signal (Tableau 12) : les deux agents, partageant un même état, peuvent collaborer implicitement pour faire osciller la balle entre les deux buts plutôt que de chercher à marquer. La reformulation *zero-sum* garantit qu'un gain de récompense pour un agent correspond nécessairement à une perte équivalente pour son adversaire, éliminant la possibilité d'une telle stratégie dégénérée.

9.6.3 Phase III – Perfectionnement mécanique

La troisième phase vise le perfectionnement des mécaniques avancées du jeu une fois les fondamentaux acquis : gestion fine du boost en fonction de l'adversaire (*BoostKeepReward_ZS*, également formulée en *zero-sum* pour les mêmes raisons que ci-dessus), optimisation du kickoff (*KickoffReward*), conservation de l'énergie cinétique (*EnergyReward*) et maîtrise du jeu aérien à distance (*AerialDistanceReward*). Le poids du signal de but est augmenté par rapport à la phase précédente afin de renforcer la pression vers l'objectif final à mesure que l'agent maîtrise les fondamentaux.

Le choix d'augmenter le poids de la récompense de but à ce stade du curri-

culum repose sur le fait que l'ajout de nouvelles récompenses intermédiaires dans cette phase (notamment *KickoffReward* et *AerialDistanceReward*) a pour effet de diluer le signal de but par rapport aux autres signaux. L'augmentation de son poids vise à compenser cette dilution et à maintenir la priorité sur l'objectif final du jeu.

Également, le passage de la gestion du boost d'une récompense individuelle à une formulation *zero-sum* dans cette phase est motivé par l'observation que, dans des situations de jeu plus avancées, la gestion du boost devient un facteur stratégique critique. Un agent qui conserve son boost au détriment de l'autre peut obtenir un avantage décisif, et la formulation *zero-sum* garantit que cette dynamique est correctement reflétée dans le signal de récompense ; elle peut également encourager l'agent à adopter des stratégies plus agressives, en allant chercher le boost adverse plutôt qu'en se contentant de conserver le sien.

Ce choix d'introduire la dimension compétitive de la gestion du boost uniquement en Phase III, plutôt que dès la Phase II, répond également à une considération pédagogique : à un stade où l'agent ne maîtrise pas encore tous les fondamentaux du jeu (déplacement, contact avec la balle, gestion du boost, score), un signal *zero-sum* portant sur la gestion stratégique du boost aurait introduit un bruit prématuré dans l'apprentissage, avant que l'agent ne soit en mesure d'en tirer parti. L'introduction progressive de cette dimension stratégique, une fois les fondamentaux acquis, permet ainsi de préparer l'agent à des situations de jeu plus complexes sans nuire à l'apprentissage des comportements de base.

9.6.4 Limites d'implémentation rencontrées avec SAC

L'implémentation de SAC utilisée (rlgym-sac, Section 7.5.5) s'est révélée significativement moins mature que son équivalent PPO (rlgym-ppo, Section 7.5.4) sur plusieurs aspects critiques, identifiés par inspection directe du code source :

- **Absence de gradient clipping.** Aucun mécanisme de limitation de la norme des gradients n'est appliqué lors des mises à jour de l'acteur, des critiques ou du coefficient d'entropie. En conséquence, les poids

de récompense présentés dans le tableau 13 (calibrés pour PPO) ont provoqué une explosion numérique des gradients (perte et valeurs Q divergeant vers l’infini) dès les premières centaines de milliers de pas d’entraînement avec SAC.

- **Standardisation des retours non fonctionnelle.** Contrairement à PPO, où la standardisation des retours est effectivement appliquée au calcul de la fonction de perte du critique, l’implémentation SAC testée ne calcule cette statistique qu’à des fins de journalisation, sans jamais l’appliquer à l’entraînement. Les récompenses utilisées par SAC restent donc à une échelle brute, distincte de celle utilisée par PPO.
- **Cible d’entropie fixe.** La cible d’entropie ($\mathcal{H}_{\text{target}} = -|\mathcal{A}|$), qui détermine la pression d’exploration maintenue par l’auto-ajustement du coefficient α , est codée en dur dans l’implémentation et n’est pas paramétrable.

Ces limites ont nécessité un ajustement empirique des hyperparamètres (réduction du taux d’apprentissage, réduction de l’échelle des récompenses) avant d’obtenir un entraînement numériquement stable, et constituent en elles-mêmes une limite à la stricte comparabilité des deux entraînements.

Elles n’ont pas été corrigées dans le cadre de ce travail, dont l’objectif est de comparer empiriquement le comportement de PPO et SAC sur cet environnement plutôt que de développer ou maintenir l’implémentation logicielle de SAC elle-même. Leur correction (ajout d’un mécanisme de gradient clipping, application effective de la standardisation des retours au calcul de la loss du critique) constituerait néanmoins une piste claire d’amélioration pour des travaux futurs cherchant à établir une comparaison plus rigoureuse entre les deux algorithmes sur cet environnement.

9.6.5 Résultats et divergence de convergence entre PPO et SAC

Sous le curriculum de récompenses de la Phase I (Tableau 13), PPO a progressé de manière conforme aux observations rapportées en Section 9.4 : l’agent a rapidement acquis un comportement de poursuite de la balle (*ball chasing*) cohérent.

SAC, sur cette même fonction de récompense, a montré une dynamique diffé-

rente. Après une phase initiale d'amélioration des composantes de contact et de rapprochement vers la balle, une stagnation, voire une légère régression, a été observée sur plusieurs dizaines de millions de pas d'entraînement supplémentaires : le signal *VelocityPlayerToBallReward* a montré une tendance à la baisse continue, tandis que *AdvancedTouchReward* est devenu de plus en plus rare. Cette dégradation est survenue de manière concomitante à une stabilisation rapide du coefficient d'entropie α à une valeur faible ($\sim 0,15-0,22$), suggérant une réduction prématurée de la pression d'exploration avant la consolidation d'un comportement utile. En raison de cette stagnation, l'entraînement SAC a été interrompu à l'issue de la Phase I, sans extension aux Phases II et III.

9.6.6 Discussion : effet de l'espace d'actions continu vs discret

Deux facteurs, non mutuellement exclusifs, permettent d'expliquer ces résultats. D'une part, les limites d'implémentation identifiées ci-dessus – en particulier l'absence de gradient clipping et de standardisation effective des récompenses – ont nécessité des compromis sur les hyperparamètres qui s'éloignent de la configuration canonique de SAC, limitant potentiellement sa capacité à exploiter pleinement sa *sample efficiency* théorique.

D'autre part, l'espace d'actions continu utilisé par SAC, contrairement à l'espace discret curaté de PPO (Section 9.1.1), impose à l'agent l'apprentissage simultané du contrôle moteur de base – la coordination cohérente des huit dimensions continues du véhicule – et de la stratégie de jeu, alors que PPO bénéficie d'un espace d'actions déjà restreint aux 90 combinaisons mécaniquement pertinentes. Cette charge d'apprentissage supplémentaire pourrait expliquer pourquoi SAC, malgré sa *sample efficiency* théorique supérieure en contrôle continu (Haarnoja ; Zhou ; Abbeel et al., 2018), peine à dépasser la phase de contrôle moteur de base dans cet environnement.

Cette étude exploratoire ne permet donc pas de conclure à une infériorité intrinsèque de SAC pour ce type d'environnement, mais met en évidence l'importance déterminante du choix de l'espace d'actions et de la maturité de l'implémentation algorithmique dans la comparaison empirique de deux méthodes de RL. Pour ces raisons, PPO a été retenu comme algorithme principal pour l'ensemble du curriculum décrit dans la suite de ce chapitre.

9.7 Résultats avec PPO

Afin d'évaluer le niveau de jeu atteint à l'issue du curriculum, l'agent PPO a été testé de manière déterministe – c'est-à-dire en sélectionnant à chaque pas de temps l'action de probabilité maximale plutôt qu'en échantillonnant stochastiquement – contre plusieurs adversaires de référence issus de la communauté RLGym et contre des joueurs humains sur le jeu officiel, en utilisant RLBot (Section 7.5.3). L'ensemble des matchs présentés dans cette section ont été réalisés avec l'agent à 10 milliards de pas d'entraînement, correspondant à l'issue de la Phase III du curriculum. Les fichiers de replay associés sont disponibles publiquement sur la plateforme Ballchasing⁷ (David Paulino, 2026).

Matchs contre des agents de référence. Deux agents de la communauté RLGym ont été retenus comme points de comparaison, tous deux issus du tournoi de bots Rocket League 2022 (*RLBot Championship 2022*, 2022) :

- **Element**, finaliste de ce tournoi. L'agent PPO a remporté les 3 face-à-face disputés (David Paulino, 2026).
- **Necto** (Rolv-Arild, 2021), vainqueur du tournoi en 1v1, entraîné sur environ 15 milliards de pas – soit un budget supérieur à celui de l'agent produit dans ce travail. L'agent PPO a également remporté les 3 face-à-face disputés (David Paulino, 2026).

Le résultat contre Necto est particulièrement notable, bien qu'il appelle une nuance importante : Necto a été entraîné sur les trois modes de jeu compétitifs (1v1, 2v2 et 3v3), tandis que l'agent PPO de ce travail a été entraîné exclusivement en 1v1. Cette spécialisation constitue un avantage potentiel dans ce contexte précis, l'agent ayant appris à optimiser ses comportements pour ce mode de jeu particulier, là où Necto doit généraliser à plusieurs configurations. Cela étant, battre un agent de référence entraîné sur un budget presque deux fois supérieur et sur un spectre de jeu plus large reste un indicateur fort de la compétitivité de l'agent produit, et constitue à ce titre une validation externe solide du niveau atteint.

7. Plateforme permettant d'analyser les statistiques détaillées de matchs Rocket League à partir des fichiers de replay officiels du jeu.

Matches contre des joueurs humains. L’agent a également été testé contre des joueurs humains de niveau Or à Champion en mode 2v2. Ces joueurs, bien que ne pratiquant pas spécifiquement le 1v1, représentent un niveau de jeu compétitif solide dans le contexte du jeu officiel. L’agent PPO a remporté une large majorité de ces matches, confirmant un niveau de jeu compétitif dans des conditions proches du jeu réel.

Analyse qualitative du comportement. Au-delà des scores, l’observation directe du comportement de l’agent en conditions de match, complétée par l’analyse des statistiques de replay disponibles sur Ballchasing (David Paulino, 2026), a mis en évidence plusieurs points forts et faiblesses résiduelles.

Gestion du boost. Un élément particulièrement remarquable concerne la gestion du boost. L’analyse des données de replay révèle deux comportements distincts de l’agent par rapport aux joueurs humains de niveau comparable. D’une part, l’agent fait un usage plus économe du boost à haute vitesse : là où un joueur humain tend à maintenir le boost activé même lorsque le véhicule a déjà atteint sa vitesse maximale – dépensant ainsi du boost sans gain de vitesse effectif – l’agent a appris à couper le boost dès que l’accélération supplémentaire ne produit plus d’effet utile (Figure 6). D’autre part, l’agent collecte davantage de petits boost répartis sur le terrain, privilégiant une gestion continue et distribuée de ses réserves plutôt qu’une dépendance aux grands boost fixes (Figure 7). Ces comportements sont vraisemblablement liés à l’optimisation de *BoostKeepReward_ZS* et *EnergyReward* introduits en Phase III, bien qu’une attribution causale stricte reste difficile à établir formellement compte tenu de l’interaction complexe entre l’ensemble des signaux du curriculum. Ils témoignent néanmoins d’une compréhension implicite de la mécanique de boost significativement plus fine que celle observée chez les joueurs humains testés.

Points forts et faiblesses résiduelles. Parmi les autres points forts identifiés : des situations de 50/50 offensifs bien maîtrisées et une rapidité de réaction globalement supérieure à celle des joueurs humains testés.

④ Boost used while supersonic

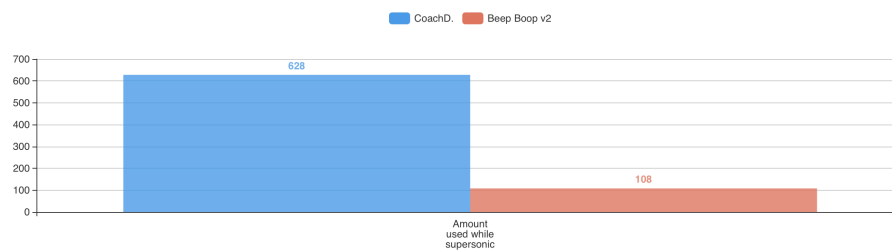


FIGURE 6 – Consommation du boost lorsque la vitesse supersonique est atteinte

Collected pads

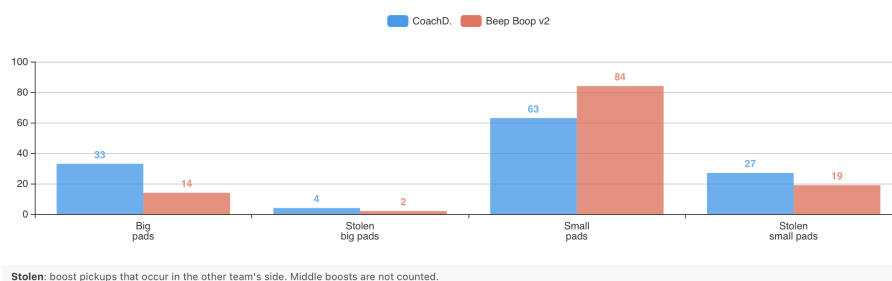


FIGURE 7 – Collecte du boost sur le terrain

L'analyse du temps passé au sol et dans les airs (Figure 8) révèle que l'agent passe la quasi-totalité du match au sol, avec un recours au jeu aérien très limité. Ce comportement suggère que les poids associés aux récompenses liées au jeu aérien (*AerialDistanceReward*, *InAirReward*) n'ont pas exercé une pression suffisante pour que l'agent explore et consolide des trajectoires aériennes complexes au cours de l'entraînement. Il en résulte une faiblesse structurelle dans les situations nécessitant une intervention aérienne rapide, notamment face à des balles hautes.

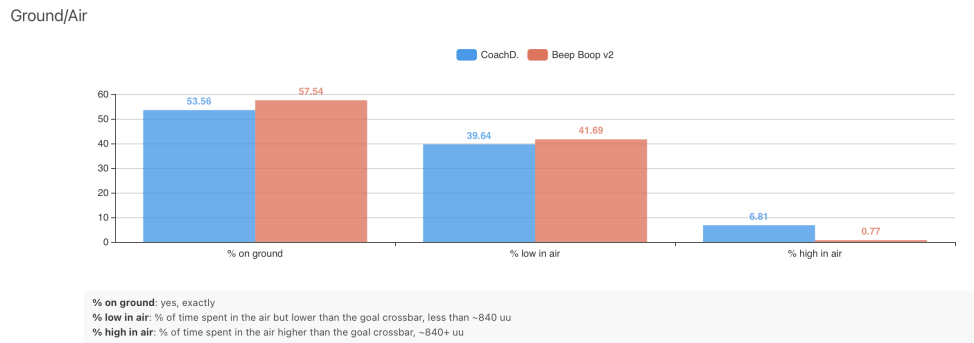


FIGURE 8 – Comparaison du temps passé au sol et dans les airs par l'agent PPO au cours d'un match contre un joueur humain de niveau Champion

Par ailleurs, l'agent manifeste une tendance à laisser l'adversaire progresser dans sa propre moitié de terrain lorsque celui-ci est en possession de la balle, plutôt que de chercher à le presser activement. Ce comportement défensif passif constitue une faiblesse identifiée, mais peut également se révéler un atout dans certaines situations : en conservant sa vitesse et son boost, l'agent se positionne avantageusement pour des contre-attaques rapides, comme le suggère la heatmap de positionnement (Figure 9), qui compare la présence sur le terrain de l'agent et de l'adversaire humain, et met en évidence une présence marquée de l'agent dans les zones de transition entre défense et attaque.

Utilisation du dérapage. Un comportement émergent particulièrement intéressant concerne l'utilisation du dérapage (*powerslide*). Comme le montre la Figure 10, l'agent y recourt de manière significative et systématique, notamment pour ajuster sa trajectoire lors de changements de direction rapides

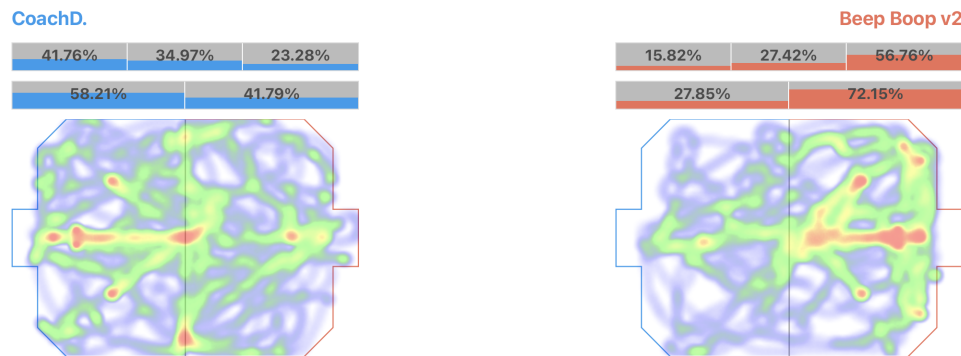


FIGURE 9 – Heatmap de positionnement comparée entre l’adversaire humain (à gauche) et l’agent PPO (à droite) au cours d’un match contre un joueur humain de niveau Champion

ou pour maintenir une orientation optimale vers la balle. Ce comportement n’a pas été explicitement récompensé dans la fonction de récompense – aucun signal dédié au powerslide n’a été introduit dans le curriculum – et semble donc émerger naturellement de l’optimisation des signaux existants, en particulier ceux liés à la vitesse de rapprochement vers la balle et à la gestion de l’énergie cinétique.

La comparaison avec les joueurs humains testés est à cet égard instructive : si ces derniers utilisent également le dérapage, ils y recourent de manière plus occasionnelle et sur des durées plus longues. L’agent, en revanche, effectue des dérapages plus courts et plus fréquents, ce qui lui confère une plus grande agilité dans ses ajustements de trajectoire et une capacité de réorientation plus réactive – au détriment potentiel de la précision dans les situations nécessitant une stabilisation prolongée.

Ces résultats valident dans l’ensemble l’efficacité de la démarche adoptée – curriculum learning combiné à un reward shaping progressif – pour amener un agent de RL à un niveau de jeu compétitif dans un environnement aussi exigeant que Rocket League. Les comportements émergents observés (gestion fine du boost, utilisation systématique du dérapage) suggèrent par ailleurs que la conception d’une fonction de récompense bien structurée peut conduire à l’acquisition de stratégies non anticipées par le concepteur, au-delà des seuls comportements explicitement ciblés par le curriculum.

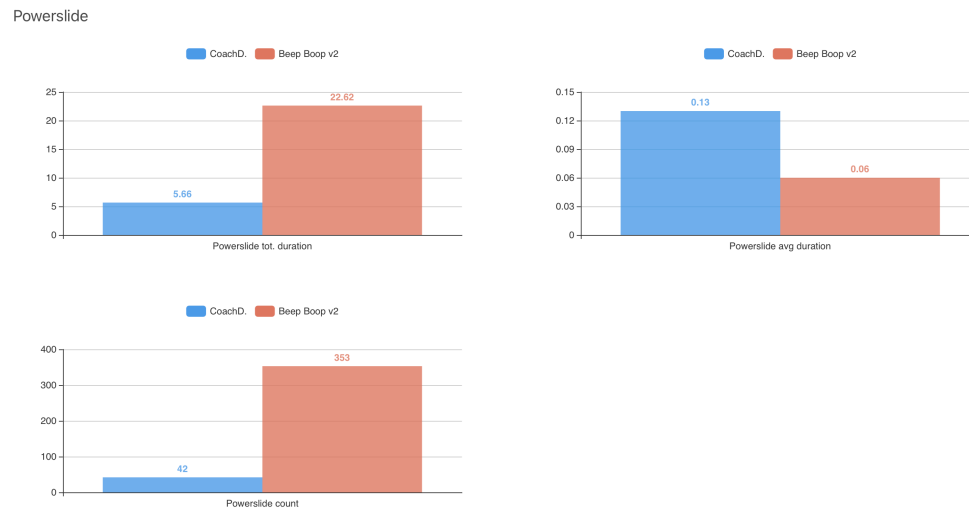


FIGURE 10 – Utilisation du dérapage par l’agent PPO et par un joueur humain de niveau Champion au cours d’un même match

10 Discussion transversale

11 Conclusion

Références

- AKIBA, Takuya; SANO, Shotaro; YANASE, Toshihiko; OHTA, Takeru; KOYAMA, Masanori, 2019. *Optuna : A Next-generation Hyperparameter Optimization Framework* [en ligne]. 2019-07-25. [visité le 04/05/2026]. Disp. à l’adr. DOI : 10.48550/arXiv.1907.10902.
- ALLEN, Matthew, 2023. *AechPro/Rlgym-Ppo* [en ligne]. [visité le 01/07/2026]. Disp. à l’adr. : <https://github.com/AechPro/rlgym-ppo>.
- AMODEI, Dario; OLAH, Chris; STEINHARDT, Jacob; CHRISTIANO, Paul; SCHULMAN, John; MANÉ, Dan, 2016. *Concrete Problems in AI Safety* [en ligne]. 2016-07-25. [visité le 03/07/2026]. Disp. à l’adr. DOI : 10.48550/arXiv.1606.06565.

- BERGSTRA, James ; BARDENET, Rémi ; BENGIO, Yoshua ; KÉGL, Balázs, 2011. Algorithms for Hyper-Parameter Optimization. In : *Proceedings of the 25th International Conference on Neural Information Processing Systems*. Red Hook, NY, USA : Curran Associates Inc., p. 2546-2554. NIPS'11. ISBN 978-1-61839-599-3.
- DAVID PAULINO, 2026. *Beep Boop v2 - Replays* [en ligne]. Ballchasing, 2026-07-01. [visit   le 02/07/2026]. Disp.    l'adr. : <https://ballchasing.com/group/beep-boop-v2-10b-tdfuhcyooj>.
- ENGSTROM, Logan ; ILYAS, Andrew ; SANTURKAR, Shibani ; TSIPRAS, Dimitris ; JANOOS, Firdaus ; RUDOLPH, Larry ; MADRY, Aleksander, 2020. *Implementation Matters in Deep Policy Gradients : A Case Study on PPO and TRPO* [en ligne]. 2020-05-25. [visit   le 03/07/2026]. Disp.    l'adr. DOI : 10.48550/arXiv.2005.12729.
- FARAMA FOUNDATION, 2025a. *Gymnasium Documentation* [en ligne]. [visit   le 04/21/2026]. Disp.    l'adr.: <https://gymnasium.farama.org/index.html>.
- FARAMA FOUNDATION, 2025b. *Gymnasium Documentation - Cliff Walking* [en ligne]. [visit   le 06/26/2026]. Disp.    l'adr.: https://gymnasium.farama.org/environments/toy_text/cliff_walking.html.
- FARAMA FOUNDATION, 2025c. *Gymnasium Documentation - Lunar Lander* [en ligne]. [visit   le 05/04/2026]. Disp.    l'adr.: https://gymnasium.farama.org/environments/box2d/lunar_lander.html.
- FUJIMOTO, Scott ; HOOFF, Herke van ; MEGER, David, 2018. *Addressing Function Approximation Error in Actor-Critic Methods* [en ligne]. 2018-10-22. [visit   le 13/05/2026]. Disp.    l'adr. DOI : 10.48550/arXiv.1802.09477.
- HAARNOJA, Tuomas ; ZHOU, Aurick ; ABBEEL, Pieter ; LEVINE, Sergey, 2018. *Soft Actor-Critic : Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor* [en ligne]. 2018-08-08. [visit   le 31/03/2026]. Disp.    l'adr. DOI : 10.48550/arXiv.1801.01290.
- HAARNOJA, Tuomas ; ZHOU, Aurick ; HARTIKAINEN, Kristian ; TUCKER, George ; HA, Sehoon ; TAN, Jie ; KUMAR, Vikash ; ZHU, Henry ; GUPTA, Abhishek ; ABBEEL, Pieter ; LEVINE, Sergey, 2019. *Soft Actor-Critic Algorithms and Applications* [en ligne]. 2019-01-29. [visit   le 11/05/2026]. Disp.    l'adr. DOI : 10.48550/arXiv.1812.05905.

- IFTIKHAR, Arta ; GHAZANFAR, Mustansar Ali ; AYUB, Mubbashir ; ALI ALAHMARI, Saad ; QAZI, Nadeem ; WALL, Julie, 2024. A Reinforcement Learning Recommender System Using Bi-Clustering and Markov Decision Process. *Expert Systems with Applications* [en ligne]. T. 237, p. 121541 [visit  le 23/02/2026]. ISSN 0957-4174. Disp.   l'adr. DOI : 10.1016/j.eswa.2023.121541.
- LILICRAP, Timothy P. ; HUNT, Jonathan J. ; PRITZEL, Alexander ; HEESS, Nicolas ; EREZ, Tom ; TASSA, Yuval ; SILVER, David ; WIERSTRA, Daan, 2015. *Continuous Control with Deep Reinforcement Learning* [en ligne]. arXiv.org, 2015-09-09. [visit  le 04/13/2026]. Disp.   l'adr. : <https://arxiv.org/abs/1509.02971v6>.
- LITTMAN, Michael L., 1994. Markov Games as a Framework for Multi-Agent Reinforcement Learning. In : COHEN, William W. ; HIRSH, Haym ( d.). *Machine Learning Proceedings 1994* [en ligne]. San Francisco (CA) : Morgan Kaufmann, p. 157-163 [visit  le 22/06/2026]. ISBN 978-1-55860-335-6. Disp.   l'adr. DOI : 10.1016/B978-1-55860-335-6.50027-1.
- MCSWAIN, Jacob, 2026. *USA-RedDragon/Rlgym-Sac* [en ligne]. [visit  le 01/07/2026]. Disp.   l'adr. : <https://github.com/USA-RedDragon/rlgym-sac>.
- MNIH, Volodymyr ; KAVUKCUOGLU, Koray ; SILVER, David ; RUSU, Andrei A. ; VENESS, Joel ; BELLEMARE, Marc G. ; GRAVES, Alex ; RIED-MILLER, Martin ; FIDJELAND, Andreas K. ; OSTROVSKI, Georg ; PETERSEN, Stig ; BEATTIE, Charles ; SADIK, Amir ; ANTONOGLOU, Ioannis ; KING, Helen ; KUMARAN, Dharshan ; WIERSTRA, Daan ; LEGG, Shane ; HASSABIS, Demis, 2015. Human-Level Control through Deep Reinforcement Learning. *Nature* [en ligne]. Vol. 518, no. 7540, p. 529–533 [visit  le 03/31/2026]. ISSN 1476-4687. Disp.   l'adr. DOI: 10.1038/nature14236.
- NARVEKAR, Sanmit ; PENG, Bei ; LEONETTI, Matteo ; SINAPOV, Jivko ; TAYLOR, Matthew E. ; STONE, Peter, 2020. *Curriculum Learning for Reinforcement Learning Domains : A Framework and Survey* [en ligne]. 2020-09-17. [visit  le 22/06/2026]. Disp.   l'adr. DOI : 10.48550/arXiv.2003.04960.
- NG, Andrew Y. ; HARADA, Daishi ; RUSSELL, Stuart J., 1999. Policy Invariance under Reward Transformations : Theory and Application to Reward Shaping. In : *Proceedings of the Sixteenth International Conference*

on Machine Learning. San Francisco, CA, USA : Morgan Kaufmann Publishers Inc., p. 278-287. Icml '99. ISBN 1-55860-612-2.

- OPENAI; BERNER, Christopher; BROCKMAN, Greg; CHAN, Brooke; CHEUNG, Vicki; DEBIAK, Przemysław; DENNISON, Christy; FARHI, David; FISCHER, Quirin; HASHME, Shariq; HESSE, Chris; JÓZEFOVICZ, Rafal; GRAY, Scott; OLSSON, Catherine; PACHOCKI, Jakub; PETROV, Michael; PINTO, Henrique P. d O.; RAIMAN, Jonathan; SALIMANS, Tim; SCHLATTER, Jeremy; SCHNEIDER, Jonas; SIDOR, Szymon; SUTSKEVER, Ilya; TANG, Jie; WOLSKI, Filip; ZHANG, Susan, 2019. *Dota 2 with Large Scale Deep Reinforcement Learning* [en ligne]. 2019-12-13. [visité le 02/07/2026]. Disp. à l'adr. DOI : 10.48550/arXiv.1912.06680.
- PAULINO, David, 2026a. *RLGym : Agent Learnt How to Make Flicks* [en ligne]. [visité le 30/06/2026]. Disp. à l'adr. : <https://www.youtube.com/watch?v=bocBIr11ZHw>.
- PAULINO, David, 2026b. *RLGym : TouchReward and VelocityBallToGoal Hacking* [en ligne]. [visité le 22/06/2026]. Disp. à l'adr. : <https://www.youtube.com/watch?v=hdgCqtCQtPc>.
- PAULINO, David, 2026c. *RLGym : TouchReward Hacking* [en ligne]. [visité le 22/06/2026]. Disp. à l'adr. : <https://www.youtube.com/watch?v=ThIveL9bChM>.
- RAFFIN, Antonin; GALLOUÉDEC, Quentin; DORMANN, Noah; GLEAVE, Adam; ANSSI; PASQUALI, Alex; ROCAMONDE, Juan; ERNESTUS, M.; HELM, Patrick; CORENTIN; SIMONINI, Thomas; ROHRER, Tobias; TIO, Sidney; TANGRI, Rohan; TOWERS, Mark; DÖRR, Tom; UNEXPLOREDTEST; JALLET, Wilson; WANG, Steven H.; TOYER, Sam; GAVRILESCU, Roland; SCHEIKL, Paul Maria; KOTHARI, Parth; KACHAIEV, Oleksii; KLAIBER, Megan; RAML, Bernhard; SCHINDLBECK, Chris; HUANG, Costa; KERR, Dominic, 2026. *DLR-RM/Stable-Baselines3 : V2.8.0 : Dropped Python 3.9, Added Python 3.13 Support, MaskablePPO Bug Fix, Default Hyperparams for Unlisted Env in the RL Zoo, Markdown Doc* [en ligne]. Zenodo. Version v2.8.0 [visité le 21/04/2026]. Disp. à l'adr. DOI : 10.5281/ZENODO.8123988.
- RAFFIN, Antonin; KOBER, Jens; STULP, Freek, 2021. *Smooth Exploration for Robotic Reinforcement Learning* [en ligne]. 2021-06-20. Version 2 [visité le 13/05/2026]. Disp. à l'adr. DOI : 10.48550/arXiv.2005.05719.

- RLBOT, 2018. *RLBot* [en ligne]. [visit  le 02/07/2026]. Disp.   l'adr. : <https://rlbot.org/>.
- RLBot Championship 2022*, 2022 [en ligne]. [visit  le 02/07/2026]. Disp.   l'adr. : <https://www.youtube.com/watch?v=XVIxZA6gFRI>.
- RLGYM, 2020a. *List of Game Values / RL Gym* [en ligne]. [visit  le 06/15/2026]. Disp.   l'adr. : https://rlgym.org/Cheatsheets/game_values.
- RLGYM, 2020b. *RL Gym* [en ligne]. [visit  le 02/07/2026]. Disp.   l'adr. : <https://github.com/RLGym/rlgym>.
- ROLV-ARILD, 2021. *Rolv-Arild/Necto* [en ligne]. [visit  le 02/07/2026]. Disp.   l'adr. : <https://github.com/Rolv-Arild/Necto>.
- SCHULMAN, John ; MORITZ, Philipp ; LEVINE, Sergey ; JORDAN, Michael ; ABBEEL, Pieter, 2018. *High-Dimensional Continuous Control Using Generalized Advantage Estimation* [en ligne]. 2018-10-20. [visit  le 13/04/2026]. Disp.   l'adr. DOI : 10.48550/arXiv.1506.02438.
- SCHULMAN, John ; WOLSKI, Filip ; DHARIWAL, Prafulla ; RADFORD, Alec ; KLIMOV, Oleg, 2017. *Proximal Policy Optimization Algorithms* [en ligne]. arXiv.org, 2017-07-20. [visit  le 03/31/2026]. Disp.   l'adr. : <https://arxiv.org/abs/1707.06347v2>.
- SILVER, David, 2015. *Lectures on Reinforcement Learning* [en ligne]. [visit  le 02/02/2026]. Disp.   l'adr. : <https://www.davidsilver.uk/teaching/>.
- SILVER, David ; SCHRITTWIESER, Julian ; SIMONYAN, Karen ; ANTONOGLOU, Ioannis ; HUANG, Aja ; GUEZ, Arthur ; HUBERT, Thomas ; BAKER, Lucas ; LAI, Matthew ; BOLTON, Adrian ; CHEN, Yutian ; LILLICRAP, Timothy ; HUI, Fan ; SIFRE, Laurent ; van den DRIESSCHE, George ; GRAEPEL, Thore ; HASSABIS, Demis, 2017. Mastering the Game of Go without Human Knowledge. *Nature* [en ligne]. Vol. 550, no. 7676, p. 354–359 [visit  le 07/02/2026]. ISSN 1476-4687. Disp.   l'adr. DOI: 10.1038/nature24270.
- SINGH, Satinder ; JAAKKOLA, Tommi ; LITTMAN, Michael L. ; SZEPESV RI, Csaba, 2000. Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms. *Machine Learning* [en ligne]. Vol. 38, no. 3, p. 287–308 [visit  le 07/02/2026]. ISSN 1573-0565. Disp.   l'adr. DOI: 10.1023/A:1007678930559.

- STABLE-BASELINES3, 2021. *Reinforcement Learning Tips and Tricks — Stable Baselines3 Documentation* [en ligne]. [visit  le 03/07/2026]. Disp.   l'adr. : https://stable-baselines3.readthedocs.io/en/master/guide/rl_tips.html.
- SUTTON, Richard S; BARTO, Andrew G, 2018. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press. ISBN 978-0-262-03924-6.
- TANG, Chen ; ABBATEMATTEO, Ben ; HU, Jiaheng ; CHANDRA, Rohan ; MART N-MART N, Roberto ; STONE, Peter, 2024. *Deep Reinforcement Learning for Robotics : A Survey of Real-World Successes* [en ligne]. 2024-09-16. [visit  le 23/02/2026]. Disp.   l'adr. DOI : 10.48550/arXiv.2408.03539.
- WATKINS, Christopher J. C. H.; DAYAN, Peter, 1992. Q-Learning. *Mach Learn* [en ligne]. Vol. 8, no. 3, p. 279–292 [visit  le 07/03/2026]. ISSN 1573-0565. Disp.   l'adr. DOI: 10.1007/BF00992698.
- WEIGHT & BIASES, 2017. *Weights & Biases : The AI Developer Platform* [en ligne]. [visit  le 21/04/2026]. Disp.   l'adr. : <https://wandb.ai/site/>.
- WILLIAMS, Ronald J., 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Mach Learn* [en ligne]. Vol. 8, no. 3, p. 229–256 [visit  le 04/13/2026]. ISSN 1573-0565. Disp.   l'adr. DOI: 10.1007/BF00992696.
- ZEALANL, 2022. *ZealanL/RocketSim* [en ligne]. [visit  le 22/04/2026]. Disp.   l'adr. : <https://github.com/ZealanL/RocketSim>.
- ZEALANL, 2024. *ZealanL/RocketSimVis* [en ligne]. [visit  le 11/06/2026]. Disp.   l'adr. : <https://github.com/ZealanL/RocketSimVis>.